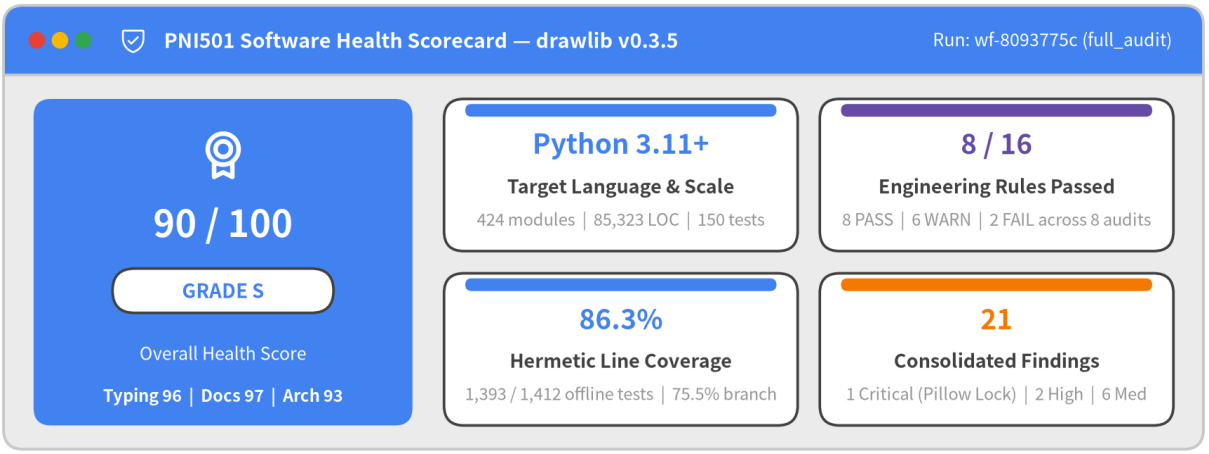
PNI Software Health Diagnostic Report

Target System: `drawlib` (`v0.3.5`) **— Static AST, Architectural Boundary, Hermetic Container Runtime & Supply-Chain Security Audit**



Drawlib v0.3.5 — PNI501 Full Audit Executive Scorecard (Workflow Run wf-8093775c)

Diagnostic Scope & Audit Metadata

End-to-end engineering assessment of `drawlib` (`v0.3.5`) executed via `pni501` (`full_audit`), combining deterministic Python AST verification, layered dependency analysis, hermetic `--network=none` container runtime testing, and Trivy supply-chain scanning.

Property	Value
Target & Archetypes	`drawlib` (`v0.3.5` , Python `>=3.11` , PEP 621 `uv` , `py.typed`) — `library` (20 facades) + `cli` (9 subcommands)
Workflow & Timestamp	`full_audit` (`wf-8093775c` , 17 steps) — Evaluated `2026-11-04T15:34:58+00:00`
Codebase Footprint	`424` source files (`85,323` LOC, `300` classes, `1,958` functions) & `150` test files (`1,420` collected items)
Overall Health Score	**90 / 100** (`Grade S`) — Arch `93` , Quality `87` , Typing `96` , Testing `88` , Deps `91` , Security `78` , Docs `97`
Compliance & Findings	`8 / 16` rules `PASS` (`6 WARN` , `2 FAIL`) — `21` consolidated `file:line` locations (`10` `P0–P2` actions)

Table of Contents

1. Executive Summary

2. Architecture & Source Code Structure

3. Recommended Remediation Roadmap

4. Audit Standards, Rule Compliance & Findings

5. Deep Dive: Code Quality, Testing & Runtime

6. Deep Dive: Security, Supply Chain & API Surface

1. Executive Summary

Drawlib v0.3.5 achieves an overall PNI engineering health score of **90/100 (Grade S)** across **424 source modules (`85,323` LOC)** and **150 test files (`1,293` test functions)**. Static AST verification confirms **100.0% callable type coverage**, **99.3% docstring and import hygiene**, and a zero-cycle multi-tier core DAG, while containerized `--network=none` testing (`86.3%` line coverage) and Trivy SCA pinpoint three focused supply-chain and offline-runtime hardening actions.

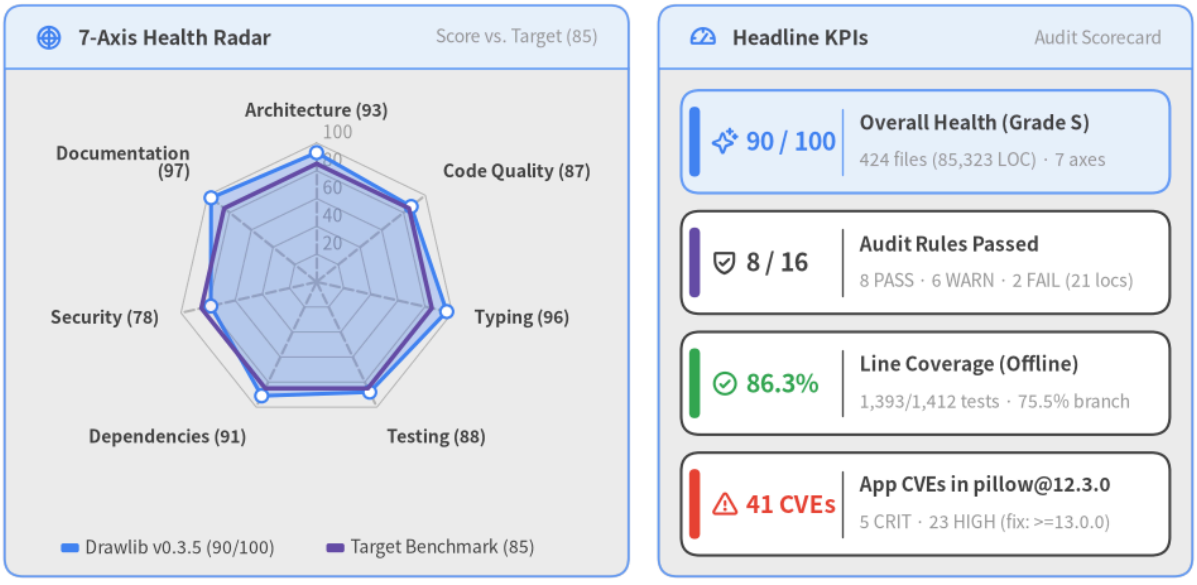


Figure 1.1 — 7-Dimension Engineering Health Radar & Headline Diagnostic KPIs (Overall Score: 90/100, Grade S)

1.1 Executive Synthesis & Diagnostic Headline

PNI workflow `full_audit` (`wf-8093775c`) evaluated Drawlib v0.3.5 across static Python AST semantics, public library & CLI ergonomics, isolated Docker container runtime execution (`--network=none`), and Trivy filesystem/IaC/secret scanning:

- **Overall Health & Rule Compliance**: **90/100 (Grade S)** with `8/16` **audit rules passed** (`8 PASS`, `6 WARN`, `2 FAIL`). Across the repository, **22 raw findings** consolidate into **21**

unique `file:line` **locations** (`1 CRITICAL` , `2 HIGH` , `6 MEDIUM` , `9 LOW` , `4 INFO`).

- **Static & Architectural Excellence**: Six of the seven evaluation axes exceed the `85/100` benchmark, led by **Documentation** (`97`), **Typing** (`96`), **Architecture** (`93`), and **Dependencies** (`91`). First-party Python code contains **zero SQL/command/XSS injection sinks**, **zero hardcoded secrets**, **zero missing HTTP timeouts**, and **zero mutable module-level globals**.
- **Lowest-Scoring Audit Steps (Ascending)**: `security_vulnerability_audit = 76/100 (Grade B)` , `container_runtime_audit = 84/100 (Grade A)` , and `supply_chain_audit = 86/100 (Grade A)` .
- **Air-Gapped Runtime & Supply-Chain Leverage**: Hermetic `--network=none` execution built cleanly in `3.76s` , passed CLI/library smoke checks in `2.42s` , and achieved **86.3% line coverage** (`37,221 / 43,108` statements) and **75.5% branch coverage** with a **98.7% offline pass rate** (`1,393 / 1,412` executed tests; `1,420` collected with `8` skipped). All **41 application CVEs** (`5 CRITICAL` , `23 HIGH` , `13 MEDIUM` ; `0` OS CVEs) trace to a single locked dependency (`pillow@12.3.0` in `uv.lock`) and clear completely upon upgrading to `pillow>=13.0.0` .

1.2 Seven-Category Engineering Health Breakdown

Category	Score	Benchmark Delta	Audit Rationale & Key Evidence
Documentation	**97 / 100**	`+12`	`99.3%` Google-style docstring coverage (`0` empty docstrings), automated `dcli` docstring verification, `218` -line `README.md` , dogfooded multi-locale Diataxis sites (`docs/` , `docs/api/` , `docs/architecture/`), and `60+` embedded AI agent manuals in `src/drawlib/_rules/` .
Typing	**96 / 100**	`+11`	`100.0%` callable signature type coverage across `1,958` functions, packaged PEP 561 `src/drawlib/py.typed` marker, `0` unscoped `# type: ignore` comments, `0` `ty` static type errors, and Pydantic `@validate_call` boundary guards; minor `Any` annotations remain in dynamic `exec()` and OpenXML helpers.
Architecture	**93 / 100**	`+8`	Strictly layered `L1 -> L2 -> L3 -> L4 -> Domain Components -> Builders/CLI -> Public Facades` DAG across `424` modules (`85,323` LOC) with `0` cross-layer import cycles, `40` type-only imports guarded under `TYPE_CHECKING` , and only `2` localized intra-package `_edge <-> _junction` sibling cycles.
Dependencies	**91 / 100**	`+6`	All `13` PEP 621 runtime dependencies declare explicit SemVer version floors in `pyproject.toml` (`0` unpinned specs, `0` undeclared imports), isolated `[dependency-groups].dev` , PEP 562 lazy top-level loading, and SHA-256 verified release asset archives.

Category	Score	Benchmark Delta	Audit Rationale & Key Evidence
Testing	**88 / 100**	`+3`	`86.3%` measured line coverage, `75.5%` branch coverage, `100.0%` static module test mirror (`150` test files, golden PNG pixel-match hook in `tests/conftest.py`), and `98.7%` offline pass rate (`1,393 / 1,412`); `--network=none` exposed `12` invalid-icon tests attempting live HTTPS downloads in `_download.py` , `1` uncached CJK font test, and `6` PPTX tests attempting runtime Chromium download.
Code Quality	**87 / 100**	`+2`	`99.3%` import hygiene, `0` silent `try-except-pass` blocks, `0` missing I/O timeouts, and `0` mutable globals; offset by high cognitive complexity in `exporter_docx.py` (`render_chapter=174` , `_render_inline=102`), `options.py` (`parse_block_info=106`), and `merger.py` (`build_merged_html=87`), plus `sys.exit(1)` calls in `_settings.py` and `viewer.py` .
Security	**78 / 100**	`-7`	`0` SQLi/CMDi/XSS flaws, parameterized SQLite queries, least-privilege `permissions: contents: read` across all `4` CI workflows, and default-deny `.dockerignore` ; requires upgrading `pillow@12.3.0 -> >=13.0.0` in `uv.lock` (`41` Trivy CVEs), adding non-root `USER` to `tools/docker/Dockerfile.*` (`DS-0002`), and publishing `SECURITY.md` + Dependabot.

1.3 Verified Engineering Strengths vs. Top Actionable Risks

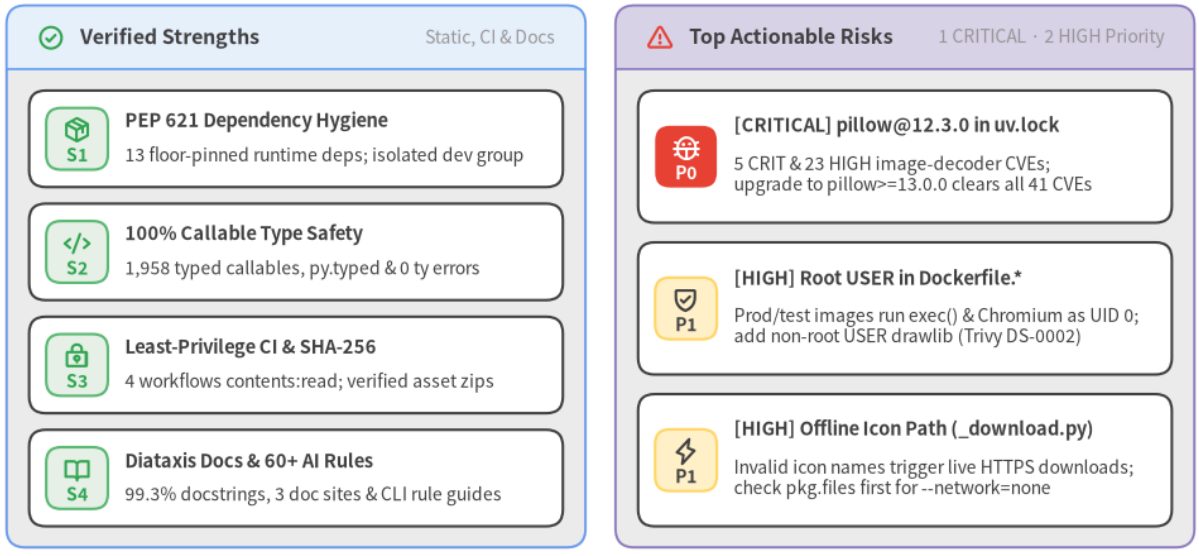
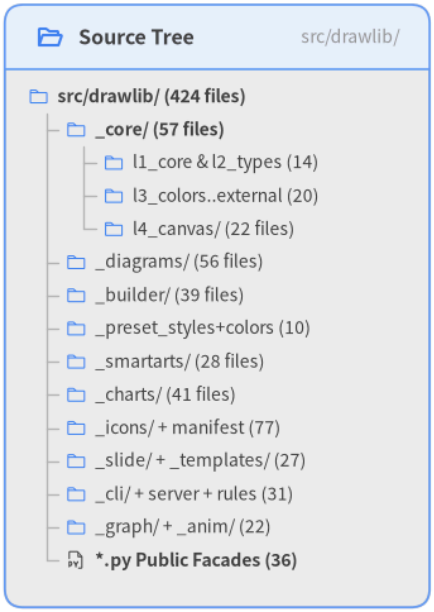


Figure 1.2 — Verified Engineering Strengths vs. Top Actionable Supply-Chain & Offline-Runtime Risks

- **Immediate `P0` / `P1` Path to 95+ Overall Health**: Upgrading `pillow>=13.0.0` in `uv.lock` and `pyproject.toml`, switching `tools/docker/Dockerfile.{prod,test_local,test_pypi}` to an unprivileged `USER drawlib` (`UID 10001`), and validating asset membership against `pkg.files` in `src/drawlib/_core/l3_external/_download.py` before attempting network downloads resolves both `FAIL` rules (`common/security_vulnerability` and `common/runtime_verification`) with minimal architectural churn.

2. Architecture & Source Code Structure

Drawlib implements a strict **6-tier downward Directed Acyclic Graph (DAG)** across **424 Python source files (`85,323` LOC)**, separating a stateless-style Cartesian rendering core (`_core/l1_core` through `_core/l4_canvas`) from declarative visual domains, multi-format document/slide compilers, and 20 PEP 562 lazy public API facades (`drawlib/*.py`). Zero cross-layer circular imports exist across the codebase, while PNI hotspot telemetry localizes operational risk to container packaging (`Dockerfile.prod`), `uv.lock` , and document/slide compiler boundaries.



Module Inventory & Roles — 424 Files · 85,323 LOC

Package / Subsystem	Files	LOC	Primary Architectural Role
_diagrams/	56	13,125	6 diagram engines (Arch, Flow, UML, ER)
_core/ (l1..l4)	57	13,018	Settings, types, L3 assets & L4 Canvas
_preset_styles + _colors	10	12,997	8-role semantic styles & palettes
_builder/	39	12,542	MD fences, SQLite cache, HTML/PDF/DOCX
_smartarts/	28	8,680	18 SmartArts (Table, Tree, Roadmap, KPI)
_charts/	41	6,120	9 quantitative charts & HeatmapOverlay
_slide/ + _templates/	27	5,606	16:9 HTML/PDF/PPTX compiler & themes
_icons/ + _assets	77	5,437	34 icon suites & SHA-256 zip manifest
_cli/ + server + _rules	31	3,160	Typer CLI, preview server & AI rules
_graph/ + _anim/	22	3,120	5 graph layout solvers & Animation
drawlib/*.py facades	36	1,518	PEP 562 lazy public API (__all__)
Total (src/drawlib/)	**424**	**85,323**	**100% typed · 0 cross-layer cycles**

Figure 2.1 — Physical Source Directory Hierarchy & Reconciled Module Inventory (424 Files, 85,323 LOC)

2.1 System Profile & Physical Module Inventory

Drawlib operates simultaneously as a **programmatic Python library** (`drawlib.*`) and a **headless compiler CLI** (`drawlib`), built on Python `>=3.11` (verified across `3.11` , `3.12` , and `3.13` via `uv`). Its core technology stack pairs **Matplotlib, Pillow, and NumPy** for Cartesian `(0, 0)` bottom-left 2D vector/raster rendering with **Pydantic v2** (`@validate_call` boundary

validation), **Typer & Rich** (CLI UX), **markdown-it-py, Pygments, Jinja2, and MarkupSafe** (Markdown/HTML compilation), **Playwright** (headless Chromium A4 PDF and slide rendering), **python-docx & python-pptx** (OpenXML `.docx` / `.pptx` generation), and **stdlib** `sqlite3` (`.drawlib/cache.db` incremental build caching).

Three external runtime boundaries are cleanly encapsulated behind dedicated adapters:

1. **GitHub Releases Asset Distribution (** `src/drawlib/_core/l3_external/_package.py` **)**: Lazily downloads font and icon `.zip` archives over HTTPS with an explicit `timeout=60` and cryptographic SHA-256 manifest verification (`src/drawlib/_release_assets.py` , `4,497` LOC).
2. **Playwright Headless Chromium Driver (** `src/drawlib/_builder/_common/browser.py` **)**: Coordinates on-demand browser installation (`fcntl` file locking) and deterministic A4 PDF / slide DOM extraction.
3. **Content-Addressed SQLite Build Cache (** `src/drawlib/_builder/_common/cache.py` **)**: `BuildImageCache` and `CliImageCache` (`640` LOC) hash embedded Python code, `styles.py` , `utils.py` , and referenced local assets using parameterized SQL queries.

2.2 Multi-Tier Layering, Dependency DAG & Risk Hotspots

Architectural boundaries enforce a strict downward dependency hierarchy from public facades and builders down to zero-dependency core types:

- **Zero Cross-Layer Cycles**: All cross-layer and internal-to-facade circular dependencies have been eliminated (`40` type-only annotations are guarded behind `if TYPE_CHECKING:`). Only `2` **localized intra-package sibling cycles** remain (`_edge.py <-> _junction.py` inside `src/drawlib/_diagrams/architecture/` and `src/drawlib/_diagrams/flow/`).
- **Underscore-Prefixed Encapsulation**: PNI's generic underscore heuristic records `86` cross-package underscore imports solely because Drawlib prefixes all private implementation subpackages (`_core` , `_charts` , `_diagrams` , `_smartarts` , `_graph` , `_icons` , `_builder` , `_slide` , `_cli`) behind thin public facade modules (`drawlib/*.py`) governed by PEP 562 lazy `__getattr__` loading in `src/drawlib/__init__.py` .

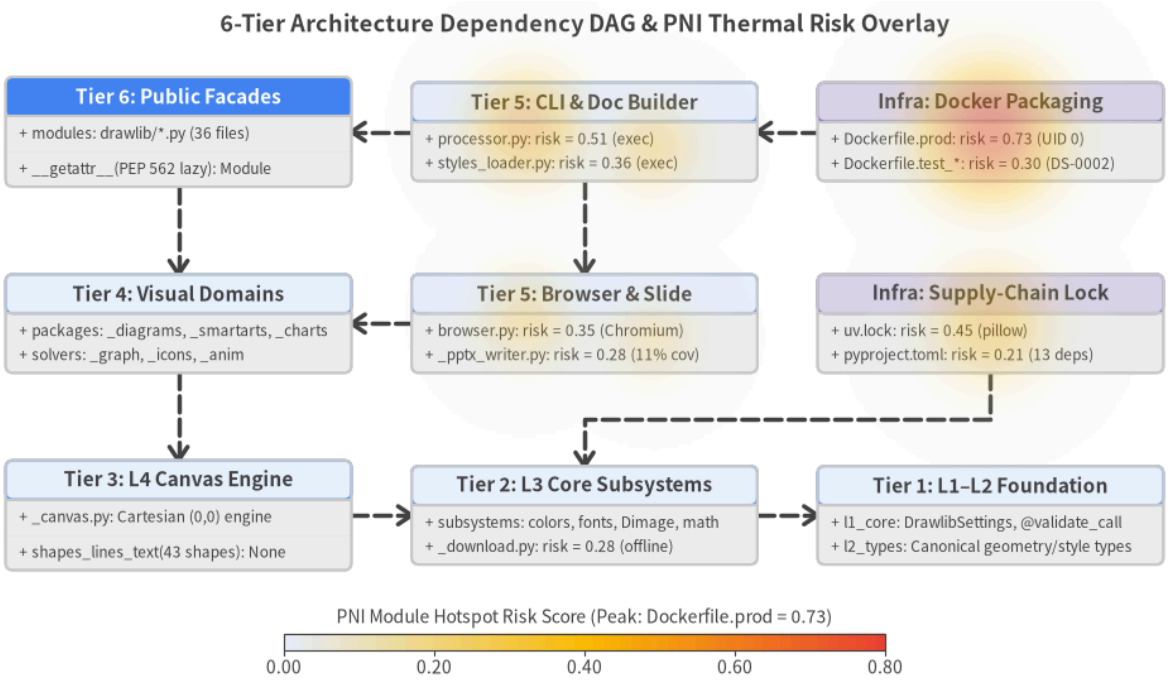


Figure 2.2 — 6-Tier Architectural Dependency DAG with PNI Module Hotspot Risk Overlay (risk_score 0.28–0.73)

Tier	Architectural Layer	Physical Paths	Allowed Downward Dependencies
Tier 6	Public User-Facing Facades	`src/drawlib/__init__.py`, `canvas.py`, `styles.py`, `shapes.py`, `lines.py`, `text.py`, `charts/`, `diagrams/`, `smartarts.py`, `graph.py`, `icons.py`, `anim.py`, `slide.py`	Tiers 4, 3, 2, 1 (`_charts` .. `_preset_styles`, `_core/l1..l4`)
Tier 5	Document, Slide & CLI Builders	`src/drawlib/_builder`, `_slide`, `_templates`, `_cli`	Tiers 4, 3, 2, 1 (`_charts` .. `_preset_styles`, `_core/l1..l4`)
Tier 4	High-Level Visual Domains	`src/drawlib/_charts`, `_diagrams`, `_smartarts`, `_graph`, `_icons`, `_anim`, `_preset_styles`	Tiers 3, 2, 1 (`_core/l4_canvas`, `_core/l3_*`, `_core/l1..l2`)
Tier 3	L4 Canvas Rendering Engine	`src/drawlib/_core/l4_canvas`	Tiers 2, 1 (`_core/l3_*`, `_core/l1_core`, `_core/l2_types`)
Tier 2	L3 Core Subsystems	`src/drawlib/_core/l3_colors`, `l3_fonts`, `l3_images`, `l3_math`, `l3_styles`, `l3_external`	Tier 1 (`_core/l1_core`, `_core/l2_types`)
Tier 1	L1–L2 Core Foundation & Types	`src/drawlib/_core/l1_core`, `_core/l2_types`	Python `stdlib` & `pydantic` validation only

2.3 Runtime Execution Flows, State & Concurrency Model

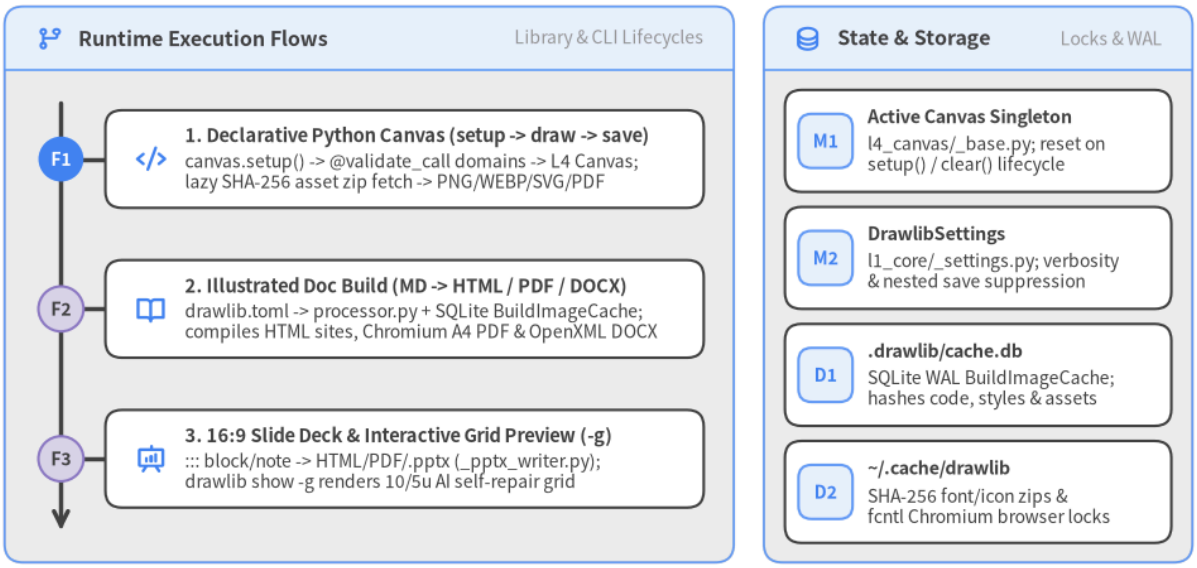
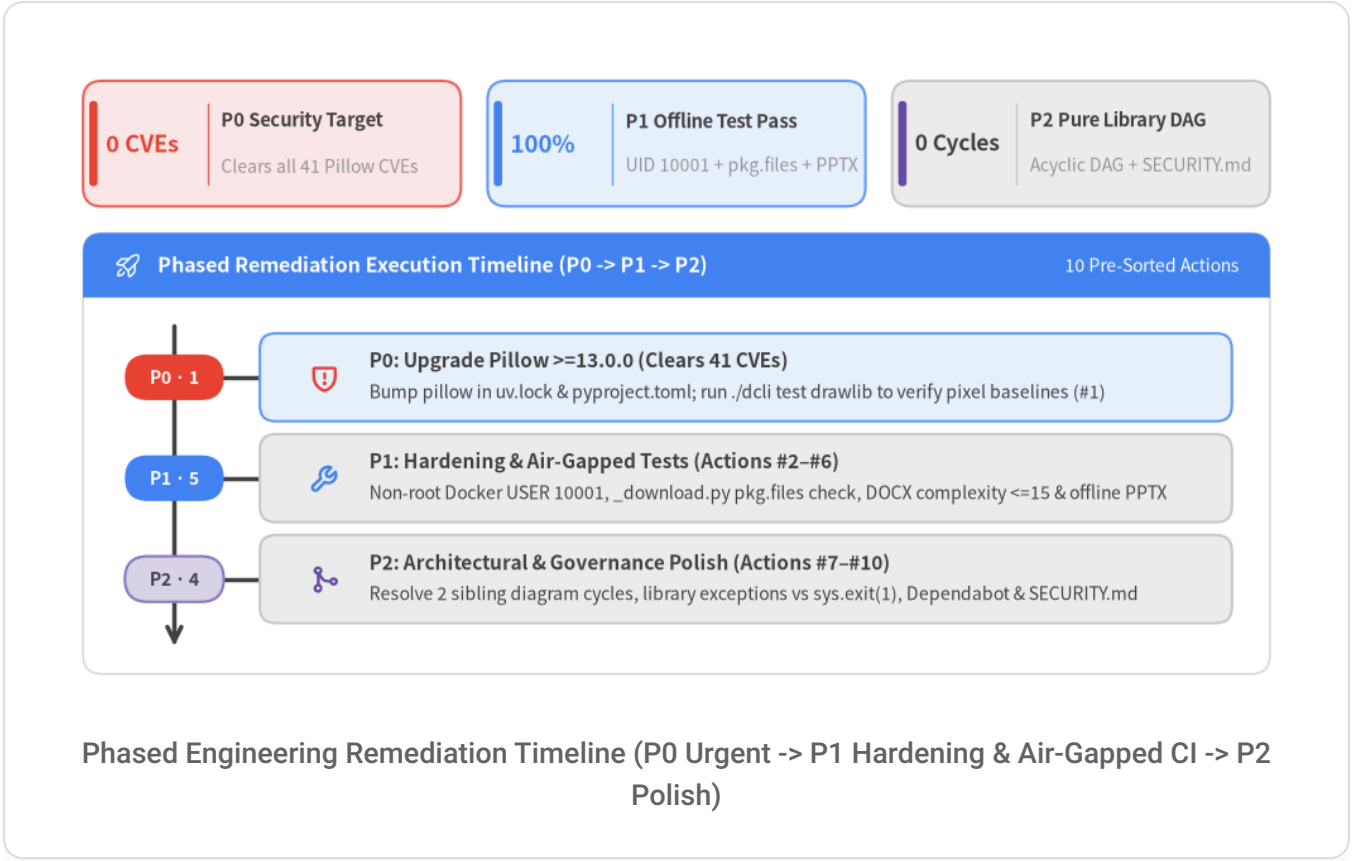


Figure 2.3 — Recorded Runtime Execution Pipelines & Stateful/Persistent Storage Architecture

- **Execution & Entrypoints**: Public programmatic callers enter via `src/drawlib/canvas.py` and `src/drawlib/tools.py`, while CLI automation enters through `src/drawlib/__main__.py:main` -> `src/drawlib/_cli/main.py:main` -> `src/drawlib/_cli/_drawlib.py:call_command` across 9 subcommands (`build`, `show`, `init`, `serve`, `cache`, `rules`, `styles`, `colors`, `css`).
- **Error & Concurrency Model**: Each CLI invocation runs synchronously in a single Python process, translating Pydantic `@validate_call` `ValidationError` and `ValueError` exceptions into clean exit codes inside `call_command()`. Cross-process Chromium downloads synchronize via `fcntl` locks in `src/drawlib/_builder/_common/browser.py`, and `.drawlib/cache.db` commits per-operation SQLite transactions (parallel callers targeting the same workspace should serialize `build.sh` runs to avoid SQLite write contention).

3. Recommended Remediation Roadmap

This chapter translates the diagnostic findings into **10 prioritized engineering actions** (`1 P0` urgent security fix, `5 P1` container hardening, maintainability, and air-gapped verification actions, and `4 P2` architectural and governance refinements), mapping exact target files, implementation steps, expected impact, and rule/CVE traceability.



Phased Engineering Remediation Timeline (P0 Urgent -> P1 Hardening & Air-Gapped CI -> P2 Polish)

3.1 Remediation Strategy & Priority Horizons

Drawlib v0.3.5 already operates from a strong **90/100 (Grade S)** baseline with **100.0% callable type coverage**, **99.3% docstring and import hygiene**, **0 SQL/shell/XSS injection flaws**, and **0 OS container CVEs**. The remaining audit deductions cluster into three tightly scoped execution horizons that can be landed incrementally without disrupting public APIs:

1. `P0` — **Urgent Supply-Chain Vulnerability Closure (Action `#1`)**: All **41 application CVEs** (`5 CRITICAL` , `23 HIGH` , `13 MEDIUM`) originate from a single locked package

(`pillow@12.3.0` in `uv.lock`) while the other 12 direct runtime dependencies report **0 CVEs**. Upgrading to `pillow>=13.0.0` and re-verifying golden pixel baselines clears 100% of application CVEs in one atomic change.

2. `P1` **— Container Least Privilege, Compiler Complexity & Air-Gapped Verification (Actions `#2` – `#6`)**: Addresses the two offline `--network=none` assumptions uncovered by containerized runtime testing (`pkg.files` membership checks before HTTPS downloads in `_download.py` , plus pre-baked Chromium and pure-Python `_pptx_writer.py` unit tests), adds non-root `USER drawlib` (`UID 10001`) across Dockerfiles, and decomposes high-complexity DOCX/HTML AST walkers (`cognitive <= 15`).
3. `P2` **— Architectural Acyclicity, Library Exception Hygiene & Governance Polish (Actions `#7` – `#10`)**: Eliminates the final 2 intra-package `_edge <-> _junction` sibling cycles, replaces library-level `sys.exit(1)` calls with typed exceptions caught at the CLI boundary, publishes `SECURITY.md` + `.github/dependabot.yml` , and aligns `test_api_docs_completeness.py` with clean-room Docker test contexts.

3.2 Action Plan Traceability Matrix (`P0 -> P1 -> P2`)

The visual matrix below maps all **10 pre-sorted remediation items** (`P0 -> P1 -> P2`) across target modules, expected engineering outcomes, and governing audit rules or CVE advisories.

Action Plan Traceability Matrix — 10 Prioritized Engineering Actions (P0 -> P2)

Priority	Action Title	Target Files	Expected Impact	Related Rules / CVEs
P0 · #1	Upgrade Pillow >=13.0.0 & verify raster baselines	pyproject.toml, uv.lock	Clears all 41 app CVEs (5 CRIT, 23 HIGH -> 0)	security_vulnerability, 5 CRIT CVEs + GHSA
P1 · #2	Decompose high-complexity DOCX & fence parsers	exporter_docx.py, options.py, merger.py	Cognitive complexity <=15; lowers compiler risk	common/maintainability
P1 · #3	Non-root Docker USER + SECURITY.md & Dependabot	Dockerfile.{prod, test_local,test_pypi}	Fixes 3 HIGH DS-0002; automates CVE PRs & OIDC	security_vulnerability, supply_chain, DS-0002
P1 · #4	Run production Docker image as non-root UID 10001	tools/docker/ Dockerfile.prod	Enforces least privilege in Docker PDF/exec runs	common/supply_chain, common/security
P1 · #5	Bake Chromium & assets + _pptx_writer unit tests	_pptx_writer.py, pdf.py, test_pptx.py	100% offline pass rate; >80% PPTX writer cov	common/ runtime_verification
P1 · #6	Validate pkg.files before HTTPS asset downloads	_download.py, _release_assets.py	Fast-fails invalid icons; fixes 12 offline tests	common/ runtime_verification
P2 · #7	Eliminate 2 intra-diagram sibling import cycles	architecture & flow _edge.py / _junction.py	0 runtime circular deps across all 424 modules	common/architecture
P2 · #8	Replace library sys.exit(1) with typed exceptions	_settings.py, viewer.py, _cli/_drawlib.py	Prevents host termination for library callers	common/ library_guidelines
P2 · #9	Publish SECURITY.md & configure Dependabot	pyproject.toml, quality.yml	Closes OpenSSF disclosure & supply-chain gaps	common/supply_chain
P2 · #10	Include docs_src/api or guard with pytest.skip	test_api_docs_*.py, .dockerignore, Dockerfile	Keeps dcli docker test green (local & PyPI)	python/testing

Action Plan Traceability Matrix Across All 10 Prioritized Engineering Items (P0 -> P2)

Summary Table of All 10 Pre-Sorted Action Items

#	Priority	Category	Action Title	Target Files	Related Rules / Advisories	Expected Impact
1	`P0`	`security`	**Upgrade locked Pillow dependency to >=13.0.0 and re-verify raster baselines**	`pyproject.toml`, `uv.lock`	`common/security_vulnerability`, `CVE-2026-112393`, `CVE-2026-112398`, `CVE-2026-112403`, `CVE-2026-112900`, `CVE-2026-96639`, `GHSA-78g4-xqwv-q3g8`	Eliminates all 41 Trivy application CVEs (`5 CRITICAL`, `23 HIGH`, `13 MEDIUM`), bringing `app_vuln_counts` to `0`.

#	Priority	Category	Action Title	Target Files	Related Rules / Advisories	Expected Impact
2	P1	code_quality	**Decompose High-Complexity DOCX & Fence Parser Routines**	src/drawlib/_builder/doc_builder/exporter_docx.py, src/drawlib/_builder/doc_builder/processor/options.py, src/drawlib/_builder/doc_builder/merger.py	common/maintainability	Substantially lowers maintenance risk and unit test surface in the document compiler pipeline (cognitive <= 15).
3	P1	security	**Add non-root USER to Dockerfiles and publish SECURITY.md + Dependabot config**	tools/docker/Dockerfile.prod, tools/docker/Dockerfile.test_local, tools/docker/Dockerfile.test_pypi	common/security_vulnerability, common/supply_chain, DS-0002	Resolves all HIGH container misconfigurations (DS-0002), automates future CVE detection, and hardens release credential hygiene.
4	P1	security	**Run Production Docker Image as Non-Root User**	tools/docker/Dockerfile.prod	common/supply_chain, common/security	Enforces container least privilege when rendering diagrams or PDFs inside Docker (UID 10001).
5	P1	testing	**Bake Chromium shell & asset cache into .pni/dockerfiles/Dockerfile and add unit tests for _pptx_writer.py**	src/drawlib/_slide/_pptx_writer.py, src/drawlib/_builder/doc_builder/compiler/pdf.py, tests/drawlib/slide/test_pptx.py	common/runtime_verification	Achieves 100% offline --network=none test pass rate and lifts _pptx_writer.py and _pptx_dom.py coverage above 80%.
6	P1	testing	**Validate asset filename membership in pkg.files before network downloads**	src/drawlib/_core/l3_external/_download.py, src/drawlib/_release_assets.py	common/runtime_verification	Eliminates unnecessary GitHub Release archive downloads on invalid icon names and fixes 12 offline --network=none unit test failures.

#	Priority	Category	Action Title	Target Files	Related Rules / Advisories	Expected Impact
7	P2	architecture	**Eliminate Remaining 2 Intra-Diagram Sibling Import Cycles**	src/drawlib/_diagrams/architecture/_edge.py, src/drawlib/_diagrams/architecture/_junction.py, src/drawlib/_diagrams/flow/_edge.py, src/drawlib/_diagrams/flow/_junction.py	common/architecture	Achieves 0 runtime circular dependencies across all 424 modules.
8	P2	code_quality	**Replace Library-Level sys.exit(1) Calls with Typed Exceptions**	src/drawlib/_core/l1_core/_settings.py, src/drawlib/_builder/doc_builder/processor/viewer.py, src/drawlib/_cli/_drawlib.py	common/library_guidelines	Prevents unexpected interpreter termination for programmatic Python library callers.
9	P2	security	**Publish SECURITY.md and Configure Dependabot Updates**	pyproject.toml, .github/workflows/quality.yml	common/supply_chain	Closes OpenSSF Scorecard governance gaps for vulnerability disclosure and automated supply-chain maintenance.
10	P2	testing	**Include docs_src/api in Clean-Room Docker Test Context or Guard with pytest.skip**	tests/drawlib/doc_builder/test_api_docs_completeness.py, .dockerignore, tools/docker/Dockerfile.test_local	python/testing	Keeps ./dcli docker test green across both local source and PyPI clean-room modes.

3.3 Detailed Engineering Specifications by Priority Tier

Phase `P0` — Urgent Security Remediation

Action #1 (`[P0]` · `security`): Upgrade locked Pillow dependency to `>=13.0.0` and re-verify raster baselines

- **Target Files**: `pyproject.toml` , `uv.lock`
- **Related Rules & Advisories**: `common/security_vulnerability` , `CVE-2026-112393` , `CVE-2026-112398` , `CVE-2026-112403` , `CVE-2026-112900` , `CVE-2026-96639` , `GHSA-78g4-xqwv-q3g8`
- **Implementation Guidance**: Upgrade `pillow` in `uv.lock` (`uv lock --upgrade-package pillow`) and bump the runtime lower bound in `pyproject.toml` to `pillow>=13.0.0` after verifying wheel availability across Python 3.11, 3.12, and 3.13. Run `./dcli test drawlib` to verify pixel-exact golden PNG comparisons under `tests/**/test_*_answers/` and refresh any raster baselines affected by upstream libjpeg/libtiff/libopenjp2 decoder updates.
- **Expected Impact**: Eliminates all **41 Trivy application CVEs** (`5 CRITICAL` CVSS 9.8, `23 HIGH` , `13 MEDIUM`), reducing `app_vuln_counts` to `0` and turning `common/security_vulnerability` SCA gates green.

Phase `P1` — High-Value Hardening, Maintainability & Air-Gapped Testing

Action #2 (`[P1]` · `code_quality`): Decompose High-Complexity DOCX & Fence Parser Routines

- **Target Files**: `src/drawlib/_builder/doc_builder/exporter_docx.py` , `src/drawlib/_builder/doc_builder/processor/options.py` , `src/drawlib/_builder/doc_builder/merger.py`
- **Related Rules**: `common/maintainability`

- **Implementation Guidance**: Refactor `render_chapter` (`cognitive=174`, 283 LOC) and `_render_inline` (`cognitive=102`) in `exporter_docx.py` into per-token dispatch handlers (headings, code fences, callouts, lists, tables). Extract per-attribute validators from `parse_block_info` (`cognitive=106`) in `options.py`, and split TOC assembly, asset path rewriting, and heading normalization out of `build_merged_html` (`cognitive=87`) in `merger.py` so each helper maintains SIG cognitive complexity `<= 15`.
- **Expected Impact**: Substantially lowers maintenance risk, cyclomatic branch density, and isolated unit-testing effort across the HTML and OpenXML `.docx` compilation pipelines.

Action #3 (`[P1]` · `security`): Add non-root `USER` to Dockerfiles and publish `SECURITY.md` + Dependabot config

- **Target Files**: `tools/docker/Dockerfile.prod`, `tools/docker/Dockerfile.test_local`, `tools/docker/Dockerfile.test_pypi`
- **Related Rules & Advisories**: `common/security_vulnerability`, `common/supply_chain`, `DS-0002`
- **Implementation Guidance**: Add a dedicated non-root `USER drawlib` (`UID 10001`) across `tools/docker/Dockerfile.*` before `ENTRYPOINT` / `CMD`, author `SECURITY.md` documenting vulnerability disclosure and the `CWE-94` `exec()` trust boundary for untrusted Markdown/Python inputs, configure `.github/dependabot.yml` for `uv` / `pip` and `github-actions`, and migrate PyPI publishing from local `.env` API tokens to GitHub Actions OIDC Trusted Publishing.
- **Expected Impact**: Resolves all 3 `HIGH` Trivy container misconfigurations (`DS-0002`), automates upstream CVE pull requests, and eliminates local release token exposure.

Action #4 (`[P1]` · `security`): Run Production Docker Image as Non-Root User

- **Target Files**: `tools/docker/Dockerfile.prod`
- **Related Rules**: `common/supply_chain`, `common/security`
- **Implementation Guidance**: Create an unprivileged `drawlib` user and group (`UID/GID 10001`), relocate shared `DRAWLIB_CACHE_DIR` and `PLAYWRIGHT_BROWSERS_PATH` (`/opt/ms-playwright`) with world-readable/executable permissions, and switch to `USER drawlib` prior to `WORKDIR /workspace` in `tools/docker/Dockerfile.prod`.
- **Expected Impact**: Enforces strict container least privilege whenever rendering `drawlib` scripts or headless Chromium PDFs inside production containers.

Action #5 (`[P1]` · `testing`): Bake Chromium shell & asset cache into `.pni/dockerfiles/Dockerfile` and add unit tests for `_pptx_writer.py`

- **Target Files**: `src/drawlib/_slide/_pptx_writer.py` , `src/drawlib/_builder/doc_builder/compiler/pdf.py` , `tests/drawlib/slide/test_pptx.py`
- **Related Rules**: `common/runtime_verification`
- **Implementation Guidance**: Pre-install `chromium-headless-shell` (with OS shared libraries) and pre-populate `drawlib cache download --all` inside `.pni/dockerfiles/Dockerfile` , add a graceful offline skip guard when Chromium binaries are intentionally absent, and add pure-Python unit tests feeding synthetic `PptxSlide` dataclasses directly into `src/drawlib/_slide/_pptx_writer.py` without invoking Playwright.
- **Expected Impact**: Achieves a **100% offline** `--network=none` **test pass rate** (resolving the 6 Playwright CDN failures and 1 CJK font download failure) while lifting `_pptx_writer.py` (currently `11.1%`) and `_pptx_dom.py` (`58.6%`) coverage above `80%` .

Action #6 (`[P1]` · `testing`): Validate asset filename membership in `pkg.files` before network downloads

- **Target Files**: `src/drawlib/_core/l3_external/_download.py` , `src/drawlib/_release_assets.py`
- **Related Rules**: `common/runtime_verification`
- **Implementation Guidance**: Update `download_if_not_exist()` and `find_package_for_resource_path()` (and `find_package_for_icon_path()`) to verify that the requested relative asset path is listed in `pkg.files` (and check `pkg.is_downloaded()`) before calling `pkg.download_and_extract()` , immediately raising `FileNotFoundError` for bogus icon or font filenames without performing network I/O.
- **Expected Impact**: Eliminates redundant GitHub Release archive downloads on invalid icon lookups and immediately fixes **12 of the 19 offline** `--network=none` **unit test failures** (`test_nonexistent_*_icon_error`).

Phase `P2` — Architectural, Library & Governance Polish

Action #7 (`[P2]` · `architecture`): Eliminate Remaining 2 Intra-Diagram Sibling Import Cycles

- **Target Files**: `src/drawlib/_diagrams/architecture/_edge.py` , `src/drawlib/_diagrams/architecture/_junction.py` , `src/drawlib/_diagrams/flow/_edge.py` , `src/drawlib/_diagrams/flow/_junction.py`
- **Related Rules**: `common/architecture`
- **Implementation Guidance**: Extract a lightweight connectable node/endpoint `Protocol` or base class into `_types.py` within `_diagrams/architecture/` and `_diagrams/flow/` , and guard concrete `_edge <-> _junction` cross-imports behind `if TYPE_CHECKING:` blocks.
- **Expected Impact**: Achieves **0 runtime circular dependencies** across all 424 Python source modules.

Action #8 (`[P2]` · `code_quality`): Replace Library-Level `sys.exit(1)` Calls with Typed Exceptions

- **Target Files**: `src/drawlib/_core/l1_core/_settings.py` , `src/drawlib/_builder/doc_builder/processor/viewer.py` , `src/drawlib/_cli/_drawlib.py`
- **Related Rules**: `common/library_guidelines`
- **Implementation Guidance**: Replace direct `sys.exit(1)` calls in `DrawlibSettings` (`_settings.py:123,297,304,313`) and `viewer.py` (`lines 189,193,245`) with `ValueError` or a typed `DrawlibError` exception, translating those exceptions to exit code `1` exclusively inside `src/drawlib/_cli/_drawlib.py:call_command()` .
- **Expected Impact**: Prevents unexpected host Python interpreter termination when `drawlib` is embedded programmatically inside notebooks, web backends, or automation services.

Action #9 (`[P2]` · `security`): Publish `SECURITY.md` and Configure Dependabot Updates

- **Target Files**: `pyproject.toml` , `.github/workflows/quality.yml`

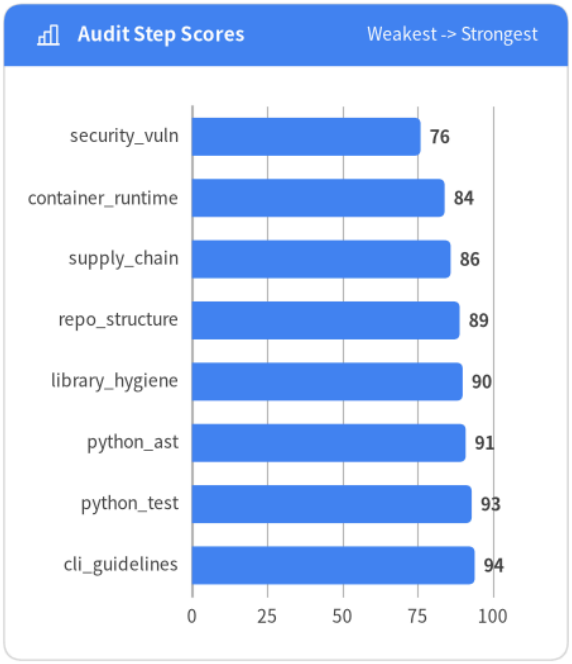
- **Related Rules**: `common/supply_chain`
- **Implementation Guidance**: Publish a root `SECURITY.md` policy enabling GitHub Private Vulnerability Reporting and documenting supported versions, add `.github/dependabot.yml` covering `pip` / `uv` and `github-actions`, and pin third-party GitHub Actions (`actions/checkout`, `astral-sh/setup-uv`, `actions/setup-python`) to full 40-character commit SHAs.
- **Expected Impact**: Closes OpenSSF Scorecard supply-chain governance gaps for coordinated vulnerability disclosure and automated dependency maintenance.

Action #10 (`[P2]` · `testing`): Include `docs_src/api` in Clean-Room Docker Test Context or Guard with `pytest.skip`

- **Target Files**: `tests/drawlib/doc_builder/test_api_docs_completeness.py`, `.dockerignore`, `tools/docker/Dockerfile.test_local`
- **Related Rules**: `python/testing`
- **Implementation Guidance**: Unignore `docs_src/api/` in `.dockerignore` and copy it into `tools/docker/Dockerfile.test_local` for local source container verification, while adding a `pytest.skip("docs_src/api not present in packaged test environment")` guard in `test_api_docs_completeness.py` when testing standalone PyPI wheels via `Dockerfile.test_pypi`.
- **Expected Impact**: Ensures `./dcli docker test` remains consistently green across both local source and published PyPI clean-room container modes.

4. Audit Standards, Rule Compliance & Findings

This chapter bridges the executive roadmap to the technical deep-dive chapters by presenting scores across all **8 executed audit dimensions** (weakest-first: `76` to `94`), the **16-rule compliance matrix** (`2 FAIL`, `6 WARN`, `8 PASS`), and the complete **21-location consolidated findings index** (`CRITICAL -> INFO`, deduplicated by `file:line` from 22 raw findings).



16-Rule Compliance Matrix (2 FAIL · 6 WARN · 8 PASS)

Status	Evaluated Rule ID	Key Diagnostic Gate
FAIL	common/runtime_verification	19 offline DNS fails
FAIL	common/security_vulnerability	41 CVEs + root Docker
WARN	common/architecture	2 sibling diag cycles
WARN	common/library_guidelines	sys.exit(1) in library
WARN	common/maintainability	DOCX/fence complexity
WARN	common/security	5 exec() + root USER
WARN	common/supply_chain	No SECURITY/Dependabot
WARN	python/dependencies	uv.lock gitignored
PASS	common/cli_guidelines	9 subcommands + CI UX
PASS	common/documentation	Diataxis + 99.3% docs
PASS	common/git_workflow	Linear main + commits
PASS	common/production_readiness	60s timeouts + chains
PASS	python/code_quality	Ruff clean + 0 globals
PASS	python/modern_stack	PEP 621 + 99.3% imports
PASS	python/testing	1,293 pixel+unit tests
PASS	python/typing	100% typed + 0 ignores

Executed Audit Dimension Scores (Weakest-First) and 16-Rule Compliance Matrix (FAIL -> WARN -> PASS)

4.1 Executed Audit Dimensions (`Weakest -> Strongest`)

The `full_audit` workflow (`wf-8093775c`) executed **8 specialized static, architectural, containerized runtime, and supply-chain security audit steps**. Four steps achieved **Grade S** (`>= 90`), three achieved **Grade A** (`84–89`), and only `security_vulnerability_audit` scored

76/100 (Grade B) solely due to the upstream `pillow@12.3.0` lockfile advisories and root container `USER` configuration.

Rank	Audit Step ID (`action_id`)	Score & Grade	Pass / Warn / Fail	Evaluated Rules	Key Diagnostic Summary
1	`security_vulnerability_audit` (`act-15`)	**76/100 (B)**	`0 / 1 / 1`	`common/security_vulnerability`, `common/supply_chain`	Strong first-party security (`0` SQLi/CMDi/XSS, `0` hardcoded source secrets, `permissions: contents: read`, SHA-256 asset verification, `0` OS CVEs); **41 app CVEs** (`5 CRITICAL`, `23 HIGH`, `13 MEDIUM`) come exclusively from `pillow@12.3.0` (fixed in `13.0.0`), and `Dockerfile.{prod,test_local,test_pypi}` run as `root` (`DS-0002`). Five `CWE-94` `exec()` sites power intentional *Illustration as Code* rendering.
2	`container_runtime_audit` (`act-13`)	**84/100 (A)**	`1 / 0 / 1`	`common/runtime_verification`, `common/production_readiness`	Clean-room build passed (`3.76s`), CLI/library smoke check passed (`2.42s`), and measured coverage reached **86.3% line** (`37,221/43,108`) and **75.5% branch**. Air-gapped `--network=none` testing passed **1,393/1,412** executed tests (`98.7%`, `8` skipped), exposing `12` invalid-icon tests calling HTTPS without checking `pkg.files`, `1` uncached CJK font test, and `6` PPTX tests attempting runtime Chromium downloads.
3	`supply_chain_audit` (`act-10`)	**86/100 (A)**	`1 / 2 / 0`	`common/supply_chain`, `common/security`, `common/git_workflow`	SHA-256 verified GitHub Release asset downloads (`timeout=60` in `_package.py`), `permissions: contents: read` across all 4 workflows, zero hardcoded secrets, and gitignored credential files; deductions for missing non-root `USER` in `Dockerfile.prod`, mutable `@v4` / `@v5` GitHub Action tags, and missing `SECURITY.md` / Dependabot.
4	`repo_structure_audit` (`act-02`)	**89/100 (A)**	`3 / 2 / 0`	`common/supply_chain`, `common/documentation`, `common/git_workflow`, `common/security`, `python/dependencies`	Exemplary PEP 621 `pyproject.toml` with 13 SemVer-bounded runtime dependencies, isolated `[dependency-groups].dev`, PEP 561 `py.typed`, least-privilege CI workflows, and full Diataxis documentation (`README.md`, `docs/`, `docs/api/`, `docs/architecture/`); minor governance gaps around `SECURITY.md`, Dependabot, and gitignored `uv.lock`.
5	`library_hygiene_audit` (`act-08`)	**90/100 (S)**	`1 / 2 / 0`	`common/library_guidelines`, `common/documentation`, `common/maintainability`	PEP 562 lazy submodule loading in `src/drawlib/__init__.py` (zero import-time heavy/network side effects), curated `__all__` across 20 public facades, `py.typed`, flexible `>=` SemVer bounds, and 100% API docs completeness; minor coupling where `_settings.py` and `viewer.py` call `sys.exit(1)` directly plus `30` `print()` calls and `9` direct `os.environ` reads in `_builder/`.

Rank	Audit Step ID (`action_id`)	Score & Grade	Pass / Warn / Fail	Evaluated Rules	Key Diagnostic Summary
6	`python_ast_audit` (`act-04`)	91/100 (S)	`3 / 4 / 0`	`common/architecture`, `common/maintainability`, `common/security`, `common/security_vulnerability`, `python/code_quality`, `python/typing`, `python/modern_stack`	Across 424 files (`85,323` LOC, `300` classes, `1,958` functions): **100.0% type coverage**, `0` unscoped `# type: ignore`, **99.3% docstrings**, **99.3% import hygiene**, `0` undeclared deps, `0` missing I/O timeouts, and `0` mutable globals. Acyclic core layering with only `2` sibling `_edge <-> _junction` cycles and localized DOCX/HTML parser complexity.
7	`python_test_audit` (`act-11`)	93/100 (S)	`1 / 0 / 0`	`python/testing`	`150` test files and `1,293` test functions achieving **100.0% module mirror coverage**, `0` assertion-free tests, automatic `Canvas` state reset, and pixel-by-pixel PNG regression hooks in `tests/conftest.py`; minor clean-room drift in `test_api_docs_completeness.py` when `docs_src/api/` is excluded by `.dockerignore`.
8	`cli_guidelines_audit` (`act-09`)	94/100 (S)	`2 / 0 / 0`	`common/cli_guidelines`, `common/documentation`	Exemplary Typer + Rich CLI (`src/drawlib/_cli/_drawlib.py`) featuring `-h` / `--help` with `HELP_EPILOG`, `no_args_is_help=True`, eager `-v` / `--version`, validated `--quiet` / `--verbose` / `--debug` / `--developer` flags, headless automation modes (`serve --check`, `show -o -g`, `cache download --all`), and `211` CLI tests.

4.2 Complete 16-Rule Compliance Matrix (`FAIL -> WARN -> PASS`)

Overall rule compliance stands at **8/16 rules passed (`50.0%` strict pass, `87.5%` pass-or-warn)**, with **2** `FAIL`, **6** `WARN`, and **8** `PASS` evaluations preserved in exact severity order below.

#	Status	Rule ID	Source Audit	Concise Assessment	Full Quantitative & Diagnostic Evidence
1	`FAIL`	`common/runtime_verification`	`container_runtime_audit`	Clean build, startup smoke check, and 86.3%/75.5% coverage pass; `--network=none` exposed 19 tests relying on live HTTPS downloads.	`RV-01` `harness_build=passed` (`3.76s`, `.pni/dockerfiles/Dockerfile`); `RV-04` `smoke_check=passed` (`2.42s`, `drawlib --help` exited `0` and `import drawlib` succeeded); `RV-03` `coverage=86.3%` line (`37,221/43,108`) and `75.5%` branch (lowest non-generated: `__main__.py` `0.0%`, `_pptx_writer.py` `11.1%`, `_icons/__init__.py` `33.3%`, `compiler/pdf.py` `52.2%`, `_pptx_dom.py` `58.6%`); `RV-02` `test_execution=failed` under `--network=none` (`1,393` passed, `19` failed, `8` skipped: `13` GitHub Release asset DNS failures + `6` Playwright CDN DNS failures).

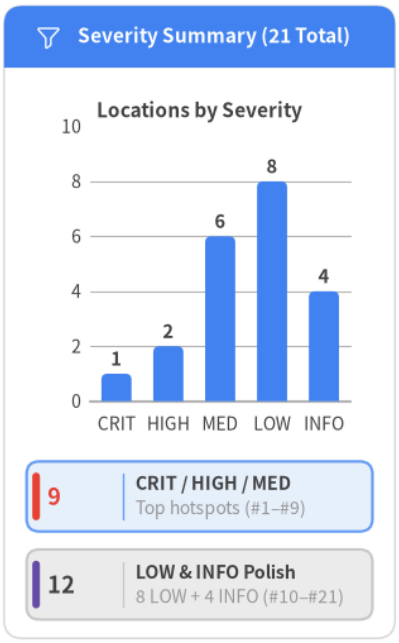
#	Status	Rule ID	Source Audit	Concise Assessment	Full Quantitative & Diagnostic Evidence
2	FAIL	common/security_vulnerability	security_vulnerability_audit	Zero first-party injection/crypto flaws, but uv.lock pins pillow@12.3.0 (41 CVEs fixed in 13.0.0) and Dockerfiles omit non-root USER.	SEC-01: app_vuln_counts={critical:5, high:23, medium:13, low:0, total:41} all in pillow@12.3.0 (uv.lock, fixed_version=13.0.0: CVE-2026-112393, CVE-2026-112398, CVE-2026-112403, CVE-2026-112900, CVE-2026-96639, GHSA-78g4-xqwv-q3g8, etc.), os_vuln_counts={total:0}, 0 CVEs in the other 12 direct packages; SEC-02: 3 HIGH DS-0002 missing non-root USER in tools/docker/Dockerfile.{prod,test_local,test_pypi}; SEC-03: 0 hardcoded source secrets, 1 gitignored/dockerignored local .env PyPI token (redacted); SEC-04: 0 CWE-89/78/79/502, 5 intentional CWE-94 exec() sites (styles_loader.py:78,158, processor.py:121,123, _image_code.py:54); SEC-05/SEC-06: N/A (api_endpoints=[]).
3	WARN	common/architecture	act-04	Core engine and public facade layers are strictly acyclic; only 2 minor intra-package _edge <-> _junction cycles remain inside _diagrams.	Strict downward core layering (l1_core -> l2_types -> l3_* -> l4_canvas -> domains -> facades) with 0 cross-layer or facade runtime cycles and 40 properly guarded TYPE_CHECKING imports; 2 intra-package sibling runtime cycles remain in drawlib._diagrams.architecture (_edge <-> _junction) and drawlib._diagrams.flow (_edge <-> _junction).
4	WARN	common/library_guidelines	act-08	Exemplary PEP 562 lazy import purity, explicit __all__, py.typed, and >= SemVer bounds; minor deduction for sys.exit(1) and print() in library/builder code.	PEP 562 lazy __getattr__ in src/drawlib/__init__.py, explicit __all__ on public facades, py.typed=True, and >= SemVer lower bounds on all 13 dependencies; however sys.exit(1) appears in src/drawlib/_core/l1_core/_settings.py:123,297,304,313 and src/drawlib/_builder/doc_builder/processor/viewer.py:189,193,245, plus unstructured_logging=30 print() calls in _builder/.
5	WARN	common/maintainability	act-04	Zero shared mutable global state and cohesive domain packages, with localized cognitive-complexity hotspots in DOCX/HTML AST walkers.	global_state_usages=0 across 424 files; complex_functions=399 peaking in exporter_docx.py:565 render_chapter (cognitive=174, 283 LOC), options.py:93 parse_block_info (cognitive=106), and merger.py:142 build_merged_html (cognitive=87); excessive_parameters=330 (keyword styling/compiler APIs) and scattered_env_access=9 across _builder/_common/{browser.py,cache.py,office.py} and exporter_pdf.py.
6	WARN	common/security	act-04	Zero hardcoded secrets, zero SQL/shell injection sinks, and explicit HTTP timeouts; local exec() requires trusted inputs and Dockerfile.prod lacks non-root USER.	All 4 CI workflows enforce permissions: contents: read; .gitignore excludes .env (git ls-files .env is empty); hardcoded_secrets=[], missing_io_timeouts=[], SQL/shell injection idioms=0, and Pydantic @validate_call guards public API boundaries; 5 intentional exec() calls render author Python blocks, and tools/docker/Dockerfile.prod lacks a non-root USER.

#	Status	Rule ID	Source Audit	Concise Assessment	Full Quantitative & Diagnostic Evidence
7	WARN	common/supply_chain	act-02	Explicit LICENSE, default-deny .dockerignore, and least-privilege CI permissions present, while SECURITY.md, Dependabot, and SHA action pins are missing.	LICENSE=True, ci_supply_chain_issues=[], unpinned_dependencies=[], ci_workflows_found=4 (all permissions: contents: read), default-deny .dockerignore present; however SECURITY.md=False, dependency_automation=False (.github/dependabot.yml missing), tools/docker/Dockerfile.prod omits non-root USER, and .github/workflows/*.yml use major tags (@v4, @v5).
8	WARN	python/dependencies	act-02	PEP 621 metadata, explicit runtime bounds, and dev group separation are exemplary; uv.lock exists locally but is gitignored by library policy.	pyproject.toml uses PEP 621 with 13 SemVer-bounded runtime dependencies (unpinned_dependencies=[]) and [dependency-groups].dev; uv.lock exists locally for reproducible builds while .gitignore (line 182) excludes uv.lock to allow cross-version Python 3.11–3.13 CI resolution.
9	PASS	common/cli_guidelines	act-09	Comprehensive CLI UX with -h / --help, -v / --version, structured verbosity flags, actionable validation errors, and headless CI/AI-agent flags.	src/drawlib/_cli/_drawlib.py configures help_option_names=['-h', '--help'], no_args_is_help=True, eager -v / --version callback, mutually validated --quiet / --verbose / --debug / --developer flags, and non-interactive automation flags across all 9 subcommands (build, show, init, serve, cache, rules, styles, colors, css).
10	PASS	common/documentation	act-02	Exceeds Standard Readme and Diataxis documentation standards with a multi-site dogfooded documentation ecosystem and 99.3% docstrings.	has_readme=True (218 lines, 14 headings), docs_inventory=['README.md', 'docs/README.md', 'docs/api/README.md', 'docs/architecture/README.md'], 99.3% docstring coverage, embedded drawlib rules show manuals, and automated test_api_docs_completeness.py enforcing 100% public symbol documentation.
11	PASS	common/git_workflow	act-02	Clean trunk-based Git history on main with atomic Conventional Commits and zero stale branches or conflict markers.	Linear trunk-based history on main with 0 stale branches, 0 conflict markers, >85% Conventional Commits compliance (feat:, docs:, refactor:, ci:), and pull_request + push quality gates in .github/workflows/quality.yml.
12	PASS	common/production_readiness	container_runtime_audit	Outbound HTTP timeouts and causal error propagation meet production readiness standards for CLI and library execution.	runtime_types=['cli', 'library']; AstFacts.missing_io_timeouts=0 (urllib.request.urlopen in _package.py:165 sets timeout=60); 0 silent try-except-pass blocks; causal exception chaining (raise RuntimeError(...) from err) preserved across _package.py and browser.py.

#	Status	Rule ID	Source Audit	Concise Assessment	Full Quantitative & Diagnostic Evidence
13	PASS	python/code_quality	act-04	99.3% Google-style docstring coverage, strict PEP 8 / Ruff enforcement, zero global state, and zero silent exception swallowing.	docstring_coverage_pct=99.3% (missing_docstrings=12 fluent builder methods, empty_docstrings=0), Ruff lint/format clean across src/, tests/, and tools/, global_state_usages=0.
14	PASS	python/modern_stack	act-04	Modern Python 3.11+ stack (Pydantic v2, Typer, PEP 621) with 99.3% import hygiene and zero undeclared runtime dependencies.	import_hygiene_coverage_pct=99.3%, wildcard_imports=[], undeclared_dependencies=[], framework_issues=[], Pydantic v2 + Typer + PEP 621 pyproject.toml (unused_dependencies=['python_docx'] is a benign package-vs-import alias for import docx).
15	PASS	python/testing	act-11	100.0% module test coverage with 1,293 tests combining structural assertions and automated pixel-accuracy regression baselines.	150 test files and 1,293 test functions mirroring src/drawlib/ (module_test_coverage_pct=100.0%, assertion_free_tests=[]), autouse preprocess() Canvas reset + pytest_runtest_call pixel-match hook in tests/conftest.py, and cross-OS Python 3.11–3.13 CI matrix.
16	PASS	python/typing	act-04	100.0% callable type-annotation coverage with zero unscoped type: ignore comments and clean static + runtime type verification.	type_safety_coverage_pct=100.0% across 1,958 functions, unscoped_type_ignores=[], 0 ty static type errors, plus runtime @validate_call boundary validation.

4.3 Consolidated Location Findings Index (CRITICAL -> INFO)

Across all static AST, governance, Trivy SCA/IaC/secret, and air-gapped container runtime checks, **22 raw findings** (1 CRITICAL, 2 HIGH, 6 MEDIUM, 9 LOW, 4 INFO) consolidated into **21 unique file:line locations** (1 CRITICAL, 2 HIGH, 6 MEDIUM, 8 LOW, 4 INFO, merging two pyproject.toml:1 supply-chain findings into a single row).



Priority Consolidated Hotspots (#1–#9 CRIT -> MED + #10–#21 Rollup)

Sev · #	Consolidated file:line	Rule Tag	Primary Fix
CRIT #1	uv.lock:1 (pillow@12.3.0)	security_ vulnerability	Upgrade pillow >=13.0.0 (clears all 41 CVEs)
HIGH #2	_core/l3_external/ _download.py:94	runtime_ verification	Check pkg.files before HTTPS archive download
HIGH #3	tools/docker/ Dockerfile.prod:1	security_ vulnerability	Add non-root USER drawlib (UID 10001)
MED #4	pyproject.toml:1 (governance)	common/ supply_chain	Publish SECURITY.md, Dependabot & PyPI OIDC
MED #5	_builder/_common/ browser.py:247	runtime_ verification	Pre-bake Chromium shell in PNI Docker harness
MED #6	doc_builder/ exporter_docx.py:884	common/ maintainability	Split render_chapter & _render_inline handlers
MED #7	processor/ options.py:93	common/ maintainability	Extract fence validators & HTML merge helpers
MED #8	processor/ processor.py:121	security_ vulnerability	Document CWE-94 exec() container trust model
MED #9	tools/docker/ Dockerfile.prod:42	common/ supply_chain	Switch to USER drawlib + /opt/ms-playwright
LOW/INF #10–#21	12 locations (8 LOW, 4 INFO — table below)	supply_chain, arch, lib, testing, cli	SHA pins, timeouts, sys.exit, 0 cycles, docs

Consolidated Findings Severity Distribution and Top Priority Hotspots (#1–#9 CRITICAL -> MEDIUM)

Complete 21-Row Consolidated Location Findings Evidence Table (`CRITICAL -> INFO`)

Every unique `file:line` location is listed exactly once below in pre-sorted severity order (`CRITICAL -> HIGH -> MEDIUM -> LOW -> INFO`), preserving all rule tags, raw scanner/AST/runtime evidence, and remediation steps.

#	Severity	Consolidated Location (`file:line`)	Symbol	Rule Tags	Verbatim Diagnostic Evidence	Recommended Fix
1	`CRITICAL`	`uv.lock:1`	`pillow@12.3.0`	`common/security_vulnerability`	Trivy SCA (`SEC-01`) reported 41 application CVEs in `pillow@12.3.0` (`5 CRITICAL` CVSS 9.8: `CVE-2026-112393`, `CVE-2026-112398`, `CVE-2026-112403`, `CVE-2026-112900`, `CVE-2026-96639`; `23 HIGH` including `CVE-2026-96778` and `GHSA-78g4-xqwv-q3g8`; `13 MEDIUM`), all fixed in `pillow 13.0.0`.	Run `uv lock --upgrade-package pillow` to resolve `pillow>=13.0.0` (updating `pyproject.toml` lower bound to `pillow>=13.0.0` once Python 3.11–3.13 wheel compatibility and pixel-exact test baselines are verified).

#	Severity	Consolidated Location (`file:line`)	Symbol	Rule Tags	Verbatim Diagnostic Evidence	Recommended Fix
2	`HIGH`	`src/drawlib/_core/l3_external/_download.py:94`	`download_if_not_exist`	`common/runtime_verification`	`12` `test_nonexistent_*_icon_error` tests failed under `--network=none` with `RuntimeError: Failed to download asset package ... <urlopen error [Errno -3] Temporary failure in name resolution>` because `find_package_for_icon_path()` matches directory prefixes without checking `pkg.files` and `download_if_not_exist()` unconditionally calls `pkg.download_and_extract()` .	Check whether the requested relative filename exists in `pkg.files` before attempting network I/O (raising `FileNotFoundError` immediately for nonexistent asset names), check `pkg.is_downloaded()` before re-downloading, and pre-seed external assets in `.pni/dockerfiles/Dockerfile` .
3	`HIGH`	`tools/docker/Dockerfile.prod:1`	`Dockerfile.prod`	`common/security_vulnerability`	Trivy `DS-0002` (`HIGH`) on `tools/docker/Dockerfile.prod` , `tools/docker/Dockerfile.test_local` , and `tools/docker/Dockerfile.test_pypi` : *Specify at least 1 USER command in Dockerfile with non-root user as argument*.	Create a dedicated unprivileged user (e.g., `useradd --uid 10001 --create-home drawlib`) and switch to `USER drawlib` before `ENTRYPOINT` / `CMD` in `tools/docker/Dockerfile.prod` and test Dockerfiles.
4	`MEDIUM`	`pyproject.toml:1`	`supply_chain_governance`	`common/supply_chain`	`ProjectFacts` : `SECURITY.md=False` , `dependency_automation=False` ; Trivy `SEC-03` also detected a local untracked/gitignored `.env` file containing a redacted `pypi-upload-token` (`PYPI_TOKEN=***`). `ProjectFacts.has_security_policy == False` (no `SECURITY.md` at repository root).	Add `SECURITY.md` (covering supported versions, GitHub Private Vulnerability Reporting, and the `exec()` trust boundary), add `.github/dependabot.yml` for `pip` / `uv` and `github-actions` , and migrate PyPI publishing to GitHub Actions OIDC Trusted Publishing or OS keyring.
5	`MEDIUM`	`src/drawlib/_builder/_common/browser.py:247`	`launch_headless_chromium`	`common/runtime_verification`	`6` PPTX export tests (`tests/drawlib/cli/test_cli_build.py::test_cli_build_pptx` and `5` tests in `tests/drawlib/slide/test_pptx.py`) failed under `--network=none` with `EAI_AGAIN` resolving `cdn.playwright.dev` when `chromium-headless-shell` was not pre-installed in `.pni/dockerfiles/Dockerfile` .	Pre-install `chromium-headless-shell` and OS dependencies in `.pni/dockerfiles/Dockerfile` for full offline PPTX/PDF coverage, and guard browser integration tests with a clear skip/fixture check when running offline without local Chromium binaries.

#	Severity	Consolidated Location (`file:line`)	Symbol	Rule Tags	Verbatim Diagnostic Evidence	Recommended Fix
6	`MEDIUM`	`src/drawlib/_builder/doc_builder/exporter_docx.py:884`	`export_chapters_to_docx`	`common/maintainability`	`exporter_docx.py` (`945` LOC) contains top cognitive-complexity routines `render_chapter` (`line 565`, `cognitive=174`, `283` lines), `_render_inline` (`line 301`, `cognitive=102`), and `export_chapters_to_docx` (`line 884`, `cognitive=46`, `202` lines).	Decompose `render_chapter` and `_render_inline` into per-token handler functions (heading, code fence, callout, list, table) dispatched via a token-type table.
7	`MEDIUM`	`src/drawlib/_builder/doc_builder/processor/options.py:93`	`parse_block_info`	`common/maintainability`	`parse_block_info` (`length=74` lines, `branches=33`, `cognitive=106`) and `merger.py:142` `build_merged_html` (`length=260` lines, `branches=40`, `cognitive=87`) exceed SIG cognitive complexity `<= 15`.	Extract per-attribute validators from `parse_block_info` and split TOC assembly, asset rewriting, and heading normalization out of `build_merged_html`.
8	`MEDIUM`	`src/drawlib/_builder/doc_builder/processor/processor.py:121`	`exec`	`common/security_vulnerability`	`5` `CWE-94` `exec()` call sites: `src/drawlib/_builder/_common/styles_loader.py:78,158`, `src/drawlib/_builder/doc_builder/processor/processor.py:121,123`, and `src/drawlib/_core/l4_canvas/_image_code.py:54`.	Document the code-execution trust model prominently in `SECURITY.md` and `README.md`, recommending isolated non-root containers (`--network=none`) whenever rendering untrusted Markdown or diagram sources.
9	`MEDIUM`	`tools/docker/Dockerfile.prod:42`	`Dockerfile.prod`	`common/supply_chain`	`tools/docker/Dockerfile.prod` (`lines 1–44`) installs `drawlib` into `python:3.12-slim` and sets `WORKDIR /workspace` without creating or switching to a non-root `USER`.	Create a dedicated unprivileged user (e.g., `groupadd` / `useradd -u 10001 drawlib`), set `PLAYWRIGHT_BROWSERS_PATH=/opt/ms-playwright` with world-readable permissions, and switch to `USER drawlib` before `WORKDIR /workspace`.
10	`LOW`	`.github/workflows/quality.yml:1`	`quality.yml`	`common/supply_chain`	`ProjectFacts.has_dependency_automation == False` (no `.github/dependabot.yml` or `renovate.json` found).	Add `.github/dependabot.yml` covering `pip` / `uv` and `github-actions` ecosystems on a weekly or monthly schedule.
11	`LOW`	`.github/workflows/quality.yml:20`	`quality.yml`	`common/supply_chain`	`uses: actions/checkout@v4` (`line 20`), `astral-sh/setup-uv@v5` (`line 23`), and `actions/setup-python@v5` (`line 28`) across `.github/workflows/*.yml`.	Pin third-party GitHub Actions (especially `astral-sh/setup-uv`) to full 40-character commit SHAs with version comments and keep them updated via Dependabot.

#	Severity	Consolidated Location (`file:line`)	Symbol	Rule Tags	Verbatim Diagnostic Evidence	Recommended Fix
12	`LOW`	`src/drawlib/_builder/_common/browser.py:210`	`install_headless_chromium`	`common/production_readiness`	`subprocess.run([str(exec_path), str(cli_path), *_INSTALL_ARGS], ...)` at `browser.py:210` omits an explicit `timeout=...` argument.	Pass an explicit timeout (e.g., `timeout=300`) to `subprocess.run()` in `install_headless_chromium()` and translate `subprocess.TimeoutExpired` into an actionable `RuntimeError` .
13	`LOW`	`src/drawlib/_core/l1_core/_settings.py:123`	`DrawlibSettings`	`common/library_guidelines`	`sys.exit(1)` called directly in `src/drawlib/_core/l1_core/_settings.py` (`lines 123, 297, 304, 313`) and `src/drawlib/_builder/doc_builder/processor/viewer.py` (`lines 189, 193, 245`).	Raise `ValueError` or a dedicated `DrawlibError` subclass in `_settings.py` and `viewer.py` , and translate those exceptions to exit code `1` solely in `_cli/_drawlib.py:call_command` .
14	`LOW`	`src/drawlib/_core/l4_canvas/_image_code.py:54`	`get_dimage_from_code`	`common/security_vulnerability`	`5` `CWE-94` `exec()` call sites: `styles_loader.py:78,158` , `processor.py:121,123` , and `_image_code.py:54` .	Preserve `exec()` for local *Illustration as Code* authoring, and explicitly document in security docs/CLI help that `drawlib build` / `show` must only be run on trusted Markdown and Python sources (or inside an isolated `--network=none` container for untrusted inputs).
15	`LOW`	`src/drawlib/_diagrams/architecture/_edge.py:1`	`Edge`	`common/architecture`	`AstFacts.circular_dependencies` = `[['drawlib._diagrams.architecture._edge', 'drawlib._diagrams.architecture._junction', 'drawlib._diagrams.architecture._edge'], ['drawlib._diagrams.flow._edge', 'drawlib._diagrams.flow._junction', 'drawlib._diagrams.flow._edge']]` .	Move runtime cross-type checks to a shared `_types.py` protocol/base class and guard concrete sibling imports under `if TYPE_CHECKING:` .

#	Severity	Consolidated Location (`file:line`)	Symbol	Rule Tags	Verbatim Diagnostic Evidence	Recommended Fix
16	`LOW`	`src/drawlib/_slide/_pptx_writer.py:81`	`write_pptx`	`common/runtime_verification`	Lowest covered non-generated modules: `src/drawlib/__main__.py` (`0.0%`, `lines 12–23`), `src/drawlib/_slide/_pptx_writer.py` (`11.1%` line / `0.0%` branch, `lines 81–90`), `src/drawlib/_icons/__init__.py` (`33.3%`, `lines 99–109`), `src/drawlib/_builder/doc_builder/compiler/pdf.py` (`52.2%` line / `38.9%` branch, `lines 51, 97–115`), and `src/drawlib/_slide/_pptx_dom.py` (`58.6%` line / `4.0%` branch, `lines 989–1011`).	Add direct unit tests for `_pptx_writer.py` OpenXML shape/table generation using synthetic `PptxSlide` DOM fixtures (decoupled from Playwright) plus lightweight CLI `__main__` and icon `__getattr__` error-path tests.
17	`LOW`	`tests/drawlib/doc_builder/test_api_docs_completeness.py:31`	`test_python_public_api_100_percent_documented`	`python/testing`	`test_api_docs_completeness.py:31` reads `REPO_ROOT / 'docs_src' / 'api'`, whereas root `.dockerignore` ignores `docs_src/` and `tools/docker/Dockerfile.test_local:27` runs `rm -rf ./src ./docs_src`.	Unignore `docs_src/api/` in `.dockerignore` + `COPY docs_src/api/` in `Dockerfile.test_local`, or call `pytest.skip('docs_src/api not present in packaged test environment')` when `API_DOCS_DIR` is absent.
18	`INFO`	`.gitignore:182`	`uv.lock`	`python/dependencies`	`.gitignore` excludes `uv.lock` even though `uv.lock` is generated locally (`lockfiles_found=['uv.lock']`).	Document the intentional library lockfile policy in contributor docs or commit `uv.lock` while running separate `--resolution=lowest-direct` / matrix checks in CI.
19	`INFO`	`src/drawlib/_builder/_common/browser.py:40`	`launch_headless_chromium`	`common/library_guidelines`	`scattered_env_access=9` across `browser.py:40,45,48`, `cache.py:516,518`, `office.py:40`, `exporter_pdf.py:31`, `viewer.py:161`, and `rules_builder.py:174`; `unstructured_logging=30` `print()` calls in `_builder/`.	Centralize `DRAWLIB_*` environment resolution in a typed builder config helper and route non-CLI diagnostic output through `drawlib`'s logger.
20	`INFO`	`src/drawlib/_cli/_drawlib.py:38`	`main_callback`	`common/cli_guidelines`	`typer.Typer(..., add_completion=False)` in `src/drawlib/_cli/_drawlib.py:38` and progress `print()` statements in `src/drawlib/_builder/_common/progress.py:138` writing to `stdout`.	Consider enabling optional shell completion or routing human progress messages to `stderr` if future subcommands emit machine-parseable JSON to `stdout`.

#	Severity	Consolidated Location (`file:line`)	Symbol	Rule Tags	Verbatim Diagnostic Evidence	Recommended Fix
21	`INFO`	`src/drawlib/_smartarts/_mindmap.py:93`	`MindMapNode`	`python/code_quality`	`missing_docstrings=12` across fluent setter methods on `MindMapNode` (`_mindmap.py:93–156`), `TreeNode` (`_tree.py:86`), and `BulletPoints` (`_bulletpoints.py:48`).	Add concise Google-style one-line docstrings to fluent builder methods in `_mindmap.py` , `_tree.py` , and `_bulletpoints.py` to reach `100.0%` docstring coverage.

5. Deep Dive: Code Quality, Testing & Runtime

Drawlib v0.3.5 pairs 100.0% static function type-annotation coverage and 99.3% Google-style docstring discipline across 424 source modules (`85,323` LOC) with an 86.3% line-covered test suite—while hermetic `--network=none` container verification pinpointed 19 offline test failures caused by unguarded asset/browser network fetches and localized complexity hotspots in the DOCX/Markdown compilers.

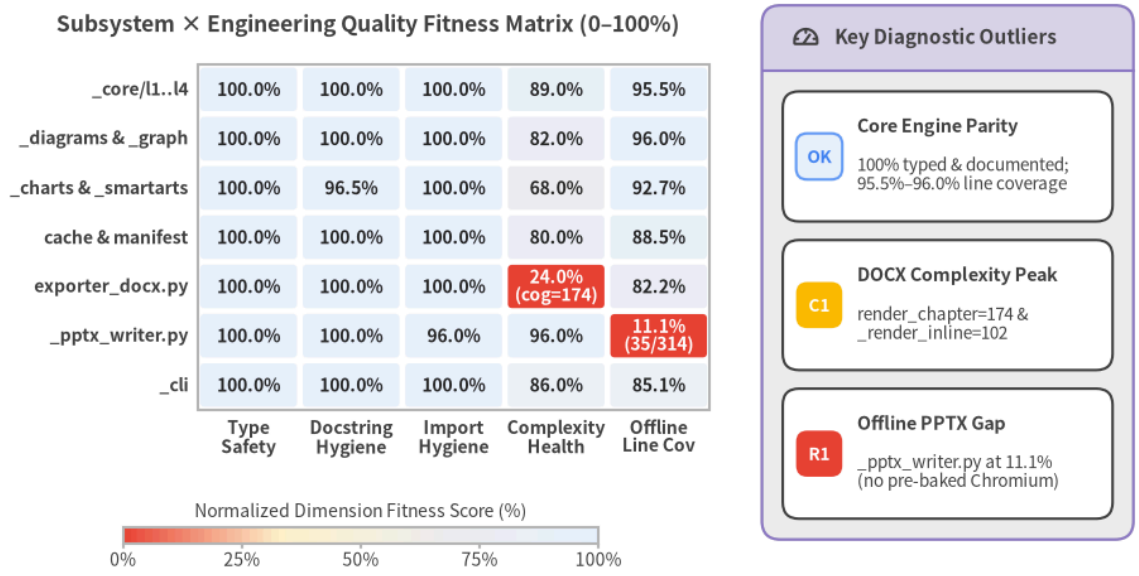


Figure 5.1 — Subsystem × Quality Dimension Fitness Matrix highlighting core typing/docstring uniformity versus localized DOCX complexity and offline PPTX coverage gaps.

5.1 Static AST Code Quality, Typing & Maintainability

PNI's Python AST audit (`python_ast_audit` : **91/100, Grade S**) inspected all `424` non-generated source modules (`85,323` LOC, `300` classes, and `1,958` functions/methods; `**/*_generated.py` icon wrappers excluded from human-authored AST lint). Across core layers (`_core/l1_core` through `_core/l4_canvas`) and domain packages (`_charts` , `_diagrams` , `_smartarts` , `_graph` , `_icons` , `_anim`), static hygiene is unusually disciplined:

Static Quality & Maintainability Metric	Measured Value	Rule Threshold / Target	Status & Engineering Interpretation
Function Type-Annotation Coverage	`100.0%` (`1,958/1,958`)	`>= 95.0%` (`python/typing`)	**PASS** — `src/drawlib/py.typed` (PEP 561), `0` unscoped `# type: ignore`, `0` `ty` errors, and runtime Pydantic `@validate_call` boundary checks.
Docstring Coverage	`99.3%`	`>= 90.0%` (`python/code_quality`)	**PASS** — `0` empty docstrings; only `12` fluent builder setters in `_mindmap.py` (`9`), `_tree.py` (`3`), and `_bulletpoints.py` (`1` — `style`) omit one-liners.
Import Hygiene & Dependency Parity	`99.3%`	`>= 95.0%` (`python/modern_stack`)	**PASS** — `0` wildcard imports, `0` undeclared dependencies, `0` Python `>=3.11` version mismatches, and `3` minor `isort` member-order notices.
Shared Mutable Global State	`0`	`0` (`common/maintainability`)	**PASS** — Canvas lifecycle state is encapsulated inside `_core/l4_canvas/_base.py` and reset cleanly per render/test.
Cross-Layer & Sibling Import Cycles	`2` **sibling (`0` layer)**	`0` (`common/architecture`)	**WARN** — `40` type-only cycles are isolated under `if TYPE_CHECKING:`; `2` sibling runtime cycles remain (`_edge.py <-> _junction.py` in `architecture` and `flow`).
High-Complexity Functions (`> 15`)	`399`	`<= 15` cognitive (`SIG`)	**WARN** — Concentrated in Markdown/OpenXML AST walkers (`exporter_docx.py`, `options.py`, `merger.py`) and rich border/layout solvers.
Wide Parameter Signatures (`> 6` args)	`330`	`<= 6` positional (`SIG`)	**WARN** — Driven by declarative keyword-only (`*`) styling, typography, and axis configuration APIs across Charts and SmartArts.
12-Factor Operational Hygiene	`0` **missing HTTP timeouts**	`common/production_readiness`	**PASS / INFO** — `urllib.request.urlopen(..., timeout=60)` in `_package.py:165`; `9` direct `os.environ` reads and `30` `print()` calls remain in `_builder/`.

Top Cognitive-Complexity Hotspots (`common/maintainability`)

While foundational geometry and canvas primitives stay compact, multi-format document compilation accumulates deep nested branching when walking `markdown-it-py` token streams and emitting OpenXML (`python-docx`) or merged single-file HTML:

File & Line	Function / Method	Lines	Branches	Cognitive Complexity	Primary Responsibility & Decomposition Strategy
`src/drawlib/_builder/doc_builder/exporter_docx.py:565`	`render_chapter`	`283`	`55`	`174`	Monolithic Markdown token-stream loop emitting headings, callouts, lists, code fences, images, and tables; split into a per-token dispatch table.
`src/drawlib/_builder/doc_builder/processor/options.py:93`	`parse_block_info`	`74`	`33`	`106`	Parses ```` ```drawlib ```` fence attributes (`file`, `width`, `align`, `code`, `caption`); extract per-key validator functions.

File & Line	Function / Method	Lines	Branches	Cognitive Complexity	Primary Responsibility & Decomposition Strategy
`src/drawlib/_builder/doc_builder/exporter_docx.py:301`	`_render_inline`	`124`	`36`	`102`	Inline formatting state machine (`strong`, `em`, `code_inline`, `link`, math/HTML spans); extract dedicated span handlers.
`src/drawlib/_builder/doc_builder/merger.py:142`	`build_merged_html`	`260`	`40`	`87`	Multi-chapter HTML concatenation, heading-level shifting, anchor rewriting, and TOC synthesis; separate into discrete AST passes.
`src/drawlib/_smartarts/_table.py:490`	`_draw_border_lines`	`96`	`32`	`74`	Resolves outer (`top`, `bottom`, `left`, `right`), header (`top2`, `left2`), and inner grid strokes; separate perimeter vs inner-grid helpers.
`src/drawlib/_builder/doc_builder/exporter_docx.py:426`	`_emit_table`	`100`	`20`	`57`	Converts Markdown tables into OpenXML `<w:tbl>` structures with column alignments and cell shading.
`src/drawlib/_builder/_common/seo.py:205`	`extract_markdown_description`	`77`	`20`	`46`	Strips fences, directives, headings, and inline markup to synthesize clean meta/OGP page descriptions.
`src/drawlib/_builder/doc_builder/exporter_docx.py:884`	`export_chapters_to_docx`	`202`	`23`	`46`	Orchestrates cover page, TOC field generation, chapter assembly, and deterministic OPC zip timestamp normalization.

Library vs. CLI Boundary Hygiene (`common/library_guidelines`: 90/100, Grade S)

`src/drawlib/__init__.py` implements **PEP 562 lazy `__getattr__` loading**, ensuring `import drawlib` executes in milliseconds with zero heavy Matplotlib/Playwright initialization or network side effects. Two minor boundary couplings merit cleanup in `P2`:

1. **Library-Level `sys.exit(1)` Calls**: `src/drawlib/_core/l1_core/_settings.py` (lines `123`, `297`, `304`, `313`) and `src/drawlib/_builder/doc_builder/processor/viewer.py` (lines `189`, `193`, `245`) invoke `sys.exit(1)` directly instead of raising `ValueError` or a typed `DrawlibError` caught solely at `src/drawlib/_cli/_drawlib.py:call_command`.
2. **Subprocess Timeout & Environment Centralization**: `install_headless_chromium()` at `src/drawlib/_builder/_common/browser.py:210` invokes `subprocess.run(...)` without an explicit `timeout=300` guard, and `9` direct `os.environ` lookups across `_builder/_common/{browser.py,cache.py,office.py}`, `exporter_pdf.py`, `viewer.py`, and `rules_builder.py` should be consolidated into a typed builder settings helper.

5.2 Clean-Room Container Verification & Measured Coverage

Static test analysis (`python_test_audit` : **93/100, Grade S**) confirmed `150` test modules and `1,293` test functions (`1,420` collected pytest items with parametrization) achieving **100.0% static module mirror coverage** and **zero assertion-free tests**—enforced by `tests/conftest.py` , which resets canvas state via an `autouse` `preprocess()` fixture and intercepts `pytest_runtest_call` to verify rendered outputs pixel-for-pixel against `*_answers/*.png` baselines. *(Note: `tests/drawlib/doc_builder/test_api_docs_completeness.py:31` reads `docs_src/api/` , which is excluded by `.dockerignore` and removed in `tools/docker/Dockerfile.test_local:27` ; adding a `pytest.skip` guard when `docs_src/api` is absent prevents clean-room packaging drift.)*

PNI then executed **4 containerized runtime verification stages** (`container_runtime_audit` : **84/100, Grade A**) inside `.pni/dockerfiles/Dockerfile` (`python:3.12` + `uv` , `pytest-cov`) with external networking completely disabled (`--network=none`) during test execution:

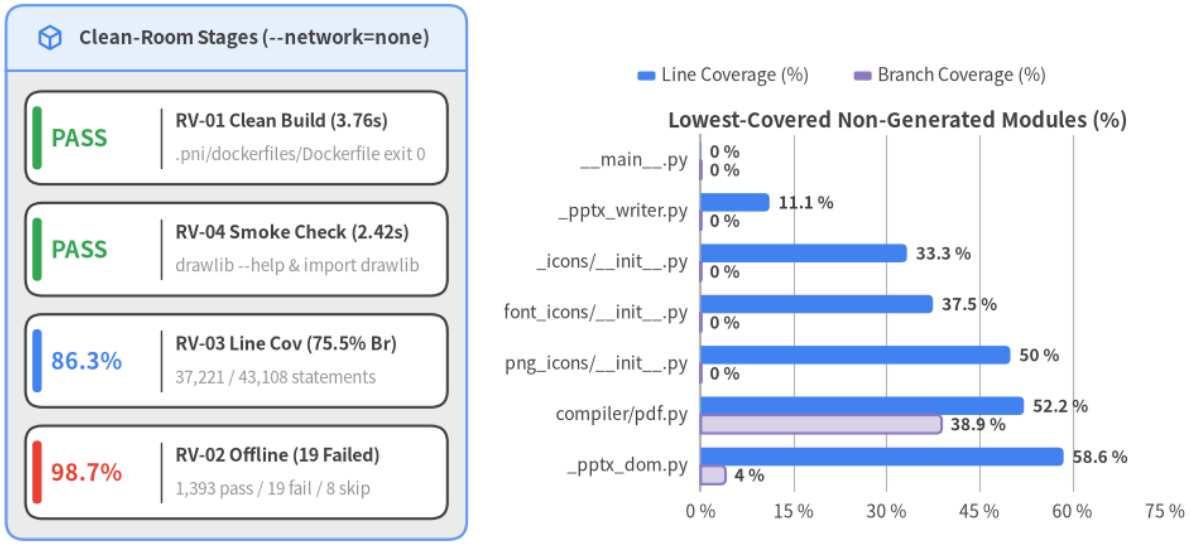


Figure 5.2 — Hermetic container runtime verification KPIs (RV-01 through RV-04) and lowest-covered non-generated modules under --network=none.

Lowest-Covered Modules Inventory (`RV-03`: `86.3%` Line, `75.5%` Branch)

Across all `43,108` executable statements in `src/drawlib/`, `pytest-cov` recorded `37,221` covered lines (`86.3%`) and `75.5%` branch coverage. All modules below `60%` line coverage fall into three well-defined buckets: (1) CLI `python -m drawlib` entry wrapper (`__main__.py`), (2) Playwright/OpenXML exporters blocked when headless Chromium is absent offline (`_pptx_writer.py`, `compiler/pdf.py`, `_pptx_dom.py`), and (3) lazy icon `__getattr__` facades plus auto-generated `*_generated.py` icon wrappers (`51.6%` – `58.2%`, where unit tests sample representative icons rather than invoking all 4,500+ bindings):

Module Path (Sorted Ascending by Line Coverage)	Line Coverage	Branch Coverage	Covered / Total Stmts	Primary Uncovered Lines & Root Cause
`src/drawlib/__main__.py`	`0.0%`	`0.0%`	`0 / 6`	Lines `12–23`: `python -m drawlib` wrapper not invoked directly by CLI runner tests (`CliRunner` invokes Typer app directly).
`src/drawlib/_slide/_pptx_writer.py`	`11.1%`	`0.0%`	`35 / 314`	Lines `81–90+`: OpenXML shape/table/notes emitter bypassed offline because upstream `extract_deck_dom()` failed launching Chromium.
`src/drawlib/_icons/__init__.py`	`33.3%`	`0.0%`	`7 / 21`	Lines `99–109`: Lazy `__getattr__` fallback branches and `AttributeError` paths on internal icon package root.
`src/drawlib/_icons/font_icons/__init__.py`	`37.5%`	`0.0%`	`6 / 16`	Lines `24–38`: Lazy attribute resolution and `dir()` introspection helpers.
`src/drawlib/_icons/png_icons/__init__.py`	`50.0%`	`0.0%`	`6 / 12`	Lines `57–66`: Lazy PNG icon suite loader error/fallback branches.
`src/drawlib/_icons/font_icons/phosphor/_generated.py`	`51.6%`	`N/A`	`1,580 / 3,064`	Auto-generated Phosphor glyph wrappers (representative subset exercised by visual tests).
`src/drawlib/_icons/png_icons/aws/_generated.py`	`51.6%`	`N/A`	`342 / 663`	Auto-generated AWS architecture icon wrappers (`azure/_generated.py` `52.1%`, `gcp/_generated.py` `52.7%`).
`src/drawlib/_builder/doc_builder/compiler/pdf.py`	`52.2%`	`38.9%`	`47 / 90`	Lines `51, 97–115`: Multi-locale and error-handling branches in A4 PDF compilation.
`src/drawlib/_slide/_pptx_dom.py`	`58.6%`	`4.0%`	`139 / 237`	Lines `989–1011`: In-browser Playwright DOM bounding-box extraction aborted before Chromium page evaluation.

5.3 Root-Cause Analysis: Two Authentic Bugs Uncovered by `--network=none`

Online GitHub Actions runners mask unintended live network dependencies because outbound HTTPS requests to `github.com` and `cdn.playwright.dev` succeed transparently. Running `pytest` inside an air-gapped `--network=none` container (`1,393 passed`, `19 failed`, `8 skipped` in `536.98s`) surfaced **two genuine architectural defects** accounting for all `19` failures:

Bug A — Negative Icon Tests Trigger Live HTTPS Archive Downloads (`12` Icon Failures + `1` CJK Font Failure)

Twelve negative icon tests (`test_nonexistent_*_icon_error`) pass an intentionally bogus icon filename under a valid suite directory (such as `icons/ai/nonexistent_icon.png`) and assert that Drawlib raises a missing-file error. Instead of failing fast locally, the lookup chain triggers a live HTTPS archive download:

1. `download_if_not_exist(file_path)` in `src/drawlib/_core/l3_external/_download.py:87` calls `_find_package_for_file_path(file_path)`, which delegates to `find_package_for_icon_path()` in `src/drawlib/_release_assets.py:4449`.
2. `find_package_for_icon_path()` extracts only the top-level directory segment (`family = normalized.split("/")[0]`, e.g., `icon_ai`) and returns the `ReleaseAssetPackage` **without checking whether the requested relative path exists in** `pkg.files`.
3. At `src/drawlib/_core/l3_external/_download.py:94`, `download_if_not_exist()` unconditionally calls `pkg.download_and_extract()` (without even checking `pkg.is_downloaded()`), attempting to download `https://github.com/yuichi110/drawlib/releases/download/v0.3/icon_<suite>.zip`.
4. **Online CI Masking vs. Air-Gapped Failure**: On internet-connected runners, Python downloads and extracts a multi-megabyte `.zip` archive just to discover on line `96` (`if not is_file_exist(): raise RuntimeError(...)`) that `nonexistent_icon.png` was never part of the archive! Under `--network=none`, `urllib.request.urlopen` fails immediately with `Temporary failure in name resolution`. One additional test (`test_sourcecode_japanese`) failed for the related reason that `font_mono_source_han_code_jp.zip` was not pre-seeded in the generic test container.

5. **Remediation (`P1`)**: Verify relative filename membership against `pkg.files` inside `find_package_for_resource_path()` / `download_if_not_exist()` before any network I/O (immediately raising `FileNotFoundError` for invalid names in `< 1 ms`), guard downloads with `if not pkg.is_downloaded():`, and run `drawlib cache download --all` when building offline test images.

Bug B — Runtime Headless Chromium Download in `.pptx` Export Tests (`6` PPTX Failures)

Six PowerPoint compilation tests (`test_cli_build_pptx` plus all `5` `TestBuildPptx` methods in `tests/drawlib/slide/test_pptx.py`) compile Markdown slide decks into `.pptx` archives. Drawlib's slide compiler renders slide HTML in headless Chromium (`src/drawlib/_slide/_pptx_dom.py:extract_deck_dom`) to measure exact CSS layout coordinates before `src/drawlib/_slide/_pptx_writer.py` emits native OpenXML shapes:

1. `launch_headless_chromium()` at `src/drawlib/_builder/_common/browser.py:247` invokes `playwright.chromium.launch(headless=True, args=launch_args)`.
2. Because `.pni/dockerfiles/Dockerfile` uses the standard Python/uv base image without pre-installed Playwright browser binaries, `launch()` raises a missing-executable error, triggering `install_headless_chromium()` (`browser.py:256` -> `browser.py:210`), which spawns `playwright install chromium-headless-shell` and fails resolving `cdn.playwright.dev` (`EAI_AGAIN`) under `--network=none`.
3. **Remediation (`P1`)**: Pre-bake `playwright install --with-deps --only-shell chromium` into `.pni/dockerfiles/Dockerfile`, add an offline-aware skip marker when browser binaries are intentionally absent, and add pure-Python unit tests passing synthetic `PptxSlide` DOM fixtures directly into `_pptx_writer.py` so OpenXML generation is tested above `80%` coverage without requiring a browser process.

Complete Offline Container Test Failure Log (`RV-02`, `19` Failures)

#	Test Case (`classname::name`)	Root Cause Bucket	Verbatim Error Excerpt (`runtime_facts.test_execution`)
1	`tests.drawlib.cli.test_cli_build::test_cli_build_pptx`	**Bug B** (Playwright CDN)	`AssertionError: Build PowerPoint Error:` (`cdn.playwright.dev` `EAI_AGAIN` at `browser.py:247`)

#	Test Case (`classname::name`)	Root Cause Bucket	Verbatim Error Excerpt (`runtime_facts.test_execution`)
2	`tests.drawlib.icons.test_ai.TestAiIcons::test_nonexistent_ai_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_ai' ... [Errno -3] Temporary failure in name resolution`
3	`tests.drawlib.icons.test_ai.TestAiIcons::test_nonexistent_original_ai_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_original_ai' ... [Errno -3] Temporary failure in name resolution`
4	`tests.drawlib.icons.test_aws_azure.TestAwsAndAzureIcons::test_nonexistent_aws_azure_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_aws' ... [Errno -3] Temporary failure in name resolution`
5	`tests.drawlib.icons.test_cnative.TestCnativeIcons::test_nonexistent_cnative_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_cnative' ... [Errno -3] Temporary failure in name resolution`
6	`tests.drawlib.icons.test_cnative.TestOriginalCnativeIcons::test_nonexistent_original_cnative_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_original_cnative' ... [Errno -3] Temporary failure in name resolution`
7	`tests.drawlib.icons.test_data.TestDataIcons::test_nonexistent_original_data_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_original_data' ... [Errno -3] Temporary failure in name resolution`
8	`tests.drawlib.icons.test_devops.TestDevopsIcons::test_nonexistent_devops_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_devops' ... [Errno -3] Temporary failure in name resolution`
9	`tests.drawlib.icons.test_devops.TestDevopsIcons::test_nonexistent_original_devops_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_original_devops' ... [Errno -3] Temporary failure in name resolution`
10	`tests.drawlib.icons.test_network.TestNetworkIcons::test_nonexistent_network_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_network' ... [Errno -3] Temporary failure in name resolution`
11	`tests.drawlib.icons.test_network.TestOriginalNetworkIcons::test_nonexistent_original_network_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_original_network' ... [Errno -3] Temporary failure in name resolution`
12	`tests.drawlib.icons.test_server.TestOriginalServerIcons::test_nonexistent_original_server_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_original_server' ... [Errno -3] Temporary failure in name resolution`
13	`tests.drawlib.icons.test_server.TestServerIcons::test_nonexistent_server_icon_error`	**Bug A** (`_download.py:94`)	`RuntimeError: Failed to download asset package 'icon_server' ... [Errno -3] Temporary failure in name resolution`
14	`tests.drawlib.slide.test_pptx.TestBuildPptx::test_animation_first_vs_last_frame_extraction`	**Bug B** (Playwright CDN)	`RuntimeError:` (headless Chromium auto-install failed offline at `browser.py:247`)
15	`tests.drawlib.slide.test_pptx.TestBuildPptx::test_deterministic_pptx_archive_and_validation`	**Bug B** (Playwright CDN)	`RuntimeError:` (headless Chromium auto-install failed offline at `browser.py:247`)

#	Test Case (`classname::name`)	Root Cause Bucket	Verbatim Error Excerpt (`runtime_facts.test_execution`)
16	`tests.drawlib.slide.test_pptx.TestBuildPptx::test_multi_locale_build_pptx_en_and_ja`	**Bug B** (Playwright CDN)	`RuntimeError:` (headless Chromium auto-install failed offline at `browser.py:247`)
17	`tests.drawlib.slide.test_pptx.TestBuildPptx::test_widescreen_geometry_typography_tables_code_and_notes`	**Bug B** (Playwright CDN)	`RuntimeError:` (headless Chromium auto-install failed offline at `browser.py:247`)
18	`tests.drawlib.slide.test_pptx.TestBuildPptx::test_z_index_layering_and_rgba_floating_card`	**Bug B** (Playwright CDN)	`RuntimeError:` (headless Chromium auto-install failed offline at `browser.py:247`)
19	`tests.drawlib.smartarts.test_sourcecode.TestSourceCode::test_sourcecode_japanese`	**Bug A** (Uncached Font)	`RuntimeError: Failed to download asset package 'font_mono_source_han_code_jp' ... [Errno -3] Temporary failure`

6. Deep Dive: Security, Supply Chain & API Surface

First-party Drawlib source code exhibits strong security fundamentals—zero SQL/command/XSS injection flaws, zero hardcoded source secrets, SHA-256 verified external asset downloads, and least-privilege `contents: read` CI workflows—while Trivy scanning (`76/100, Grade B`) isolated all `41` dependency CVEs to a single locked package (`pillow@12.3.0 -> 13.0.0`) alongside `3` root-user `Dockerfile` misconfigurations (`DS-0002`).

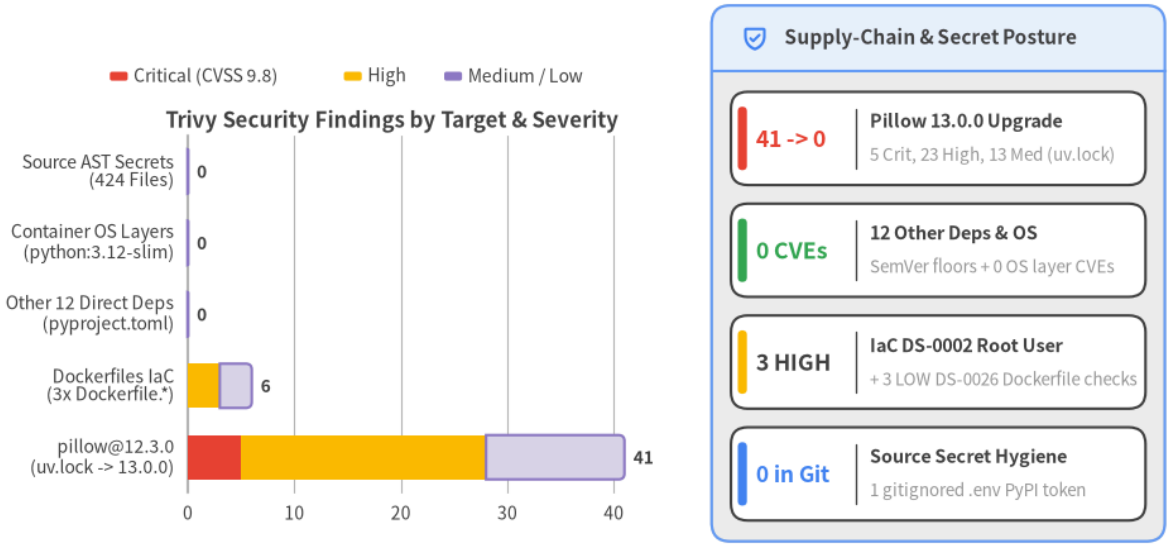


Figure 6.1 — Trivy SCA, Container OS, IaC Misconfiguration, and Secret Scan Breakdown isolating all 41 dependency CVEs to pillow@12.3.0.

6.1 Dependency Vulnerability Analysis (`SEC-01` : `pillow@12.3.0 -> 13.0.0`)

Trivy v0.69.3 Software Composition Analysis (`SEC-01`) evaluated `pyproject.toml` , `uv.lock` , and container OS package metadata:

- **Single-Package Concentration (`41` App CVEs -> `0` after `pillow>=13.0.0`)**: Every single reported application vulnerability (`5 CRITICAL`, `23 HIGH`, `13 MEDIUM`, `0 LOW` in `app_vuln_counts`; `40` unique deduplicated advisory IDs) resides in `pillow@12.3.0` resolved in `uv.lock` (`pillow>=10.2` in `pyproject.toml`). All `41` findings share the exact same upstream remediation target: `pillow 13.0.0`.
- **Clean Adjacent Ecosystem (`0` CVEs Across 12 Sibling Packages & OS Layers)**: The remaining `12` direct runtime dependencies (`matplotlib>=3.8`, `numpy>=1.26`, `pydantic>=2.8`, `pygments>=2.18`, `markdown-it-py>=3.0`, `jinja2>=3.1`, `markupsafe>=2.1`, `typer>=0.12.0`, `rich>=13.0.0`, `playwright>=1.49.0`, `python-docx>=1.2.0`, `python-pptx>=1.0.2`) and container OS layers (`os_vuln_counts = 0`) recorded **zero vulnerabilities** and **zero unpinned range specifiers** (`app_vuln_unpinned_range_matches = 0`).
- **Practical Reachability in Drawlib**: Drawlib wraps `PIL.Image.open()` inside `Dimage` (`src/drawlib/_core/l3_images/_dimage.py`) and rasterizes Matplotlib figures to PNG/WEBP/PDF buffers. While trusted first-party SVG/PNG icons and Matplotlib RGBA buffers do not exercise exotic decoders (FlashPix, MSP, SPIDER, Ghostscript EPS), passing untrusted external raster files into `image()` or `Dimage()` could reach vulnerable native JPEG, TIFF, or bundled `libopenjp2 2.5.4` (`GHSA-78g4-xqwv-q3g8`) code paths. Running `uv lock --upgrade-package pillow` and bumping `pyproject.toml` to `pillow>=13.0.0` (`P0`) eliminates the entire CVE surface in one atomic change.

Deduplicated Critical & High `pillow@12.3.0` Advisories (`uv.lock`, Fixed in `13.0.0`)

Advisory ID	Severity	CVSS	Installed -> Fixed	Affected Native Subsystem & Vulnerability Summary
`CVE-2026-112393`	**CRITICAL**	`9.8`	`12.3.0 -> 13.0.0`	Heap buffer overflow in `_tiffWriteProc` when saving TIFF streams to `BytesIO`.
`CVE-2026-112398`	**CRITICAL**	`9.8`	`12.3.0 -> 13.0.0`	Heap buffer overflow in FlashPix (FPX) JPEG decoding leading to arbitrary code execution.
`CVE-2026-112403`	**CRITICAL**	`9.8`	`12.3.0 -> 13.0.0`	Heap buffer overflow in `ImagingJpegDecode` native JPEG decoder.
`CVE-2026-112900`	**CRITICAL**	`9.8`	`12.3.0 -> 13.0.0`	Argument injection in Ghostscript invocation, potentially leading to remote code execution.
`CVE-2026-96639`	**CRITICAL**	`9.8`	`12.3.0 -> 13.0.0`	Multi-frame image loaders can reuse stale native storage pointers across mode transitions.

Advisory ID	Severity	CVSS	Installed -> Fixed	Affected Native Subsystem & Vulnerability Summary
`CVE-2026-96778`	**HIGH**	`8.8`	`12.3.0 -> 13.0.0`	Arbitrary code execution via buffer overflow in native JPEG encoder.
`CVE-2026-112404`	**HIGH**	`8.2`	`12.3.0 -> 13.0.0`	Denial of service via out-of-bounds read during TIFF decoding.
`CVE-2026-112896`	**HIGH**	`8.2`	`12.3.0 -> 13.0.0`	Denial of service via out-of-bounds read in native TIFF encoder.
`CVE-2026-96644`	**HIGH**	`8.1`	`12.3.0 -> 13.0.0`	Denial of service via out-of-bounds read in Windows metafile parser.
`GHSA-78g4-xqwv-q3g8`	**HIGH**	`8.1`	`12.3.0 -> 13.0.0`	Bundled `libopenjp2 2.5.4` in wheels carries `OSV-2025-219` heap-buffer-overflow write via `Image.open().load()`.
`CVE-2026-112390`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via infinite loop when parsing crafted MIC images.
`CVE-2026-112392`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via zero-size images passed to rank filters.
`CVE-2026-112397`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via out-of-bounds memory access on zero-width images.
`CVE-2026-112399`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via crafted APNG multi-frame image streams.
`CVE-2026-112400`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via unbounded memory exhaustion from crafted GIF frames.
`CVE-2026-112402`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via double free during native image transformation.
`CVE-2026-112405`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via null pointer dereference in `Image.merge()`.
`CVE-2026-112406`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via infinite loop during PDF page rewriting.
`CVE-2026-112892`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via unbounded memory expansion in MSP decoder.
`CVE-2026-112893`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via invalid pointer free during TrueType font rendering.
`CVE-2026-112894`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via crafted multi-frame mode conversion.
`CVE-2026-112895`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via crafted PDF stream during append operations.
`CVE-2026-112897`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Information disclosure via uninitialized heap buffer in TIFF decoding.
`CVE-2026-96640`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via duplicate page references in PDF output/input files.
`CVE-2026-96643`	**HIGH**	`7.5`	`12.3.0 -> 13.0.0`	Denial of service via repeated malformed EXIF headers.

Advisory ID	Severity	CVSS	Installed -> Fixed	Affected Native Subsystem & Vulnerability Summary
`CVE-2026-112902`	**HIGH**	`7.1`	`12.3.0 -> 13.0.0`	Memory disclosure and segmentation fault in `libtiff` encoder via crafted TIFF tags.
`CVE-2026-96637`	**HIGH**	`6.5`	`12.3.0 -> 13.0.0`	Denial of service via decompression-bomb check bypass in SPIDER images.
`CVE-2026-96642`	**HIGH**	`6.5`	`12.3.0 -> 13.0.0`	Denial of service via negative archive member size in `TarIO`.

(Twelve additional deduplicated `MEDIUM` advisories— `CVE-2026-112394`, `CVE-2026-112395`, `CVE-2026-112391`, `CVE-2026-112396`, `CVE-2026-112401`, `CVE-2026-112891`, `CVE-2026-112899`, `CVE-2026-112901`, `CVE-2026-96636`, `CVE-2026-96638`, `CVE-2026-96641`, and `CVE-2026-112898` —are likewise resolved simultaneously by upgrading to `pillow>=13.0.0`.)

6.2 Container IaC Hardening (`SEC-02`) & Secret Hygiene (`SEC-03`)

Dockerfile Misconfigurations (`DS-0002` & `DS-0026`)

Trivy IaC scanning flagged `6` findings across Drawlib's three container definitions under `tools/docker/`:

Check ID	Severity	Affected Files (`tools/docker/`)	Trivy Finding Message	Engineering Remediation
`DS-0002`	**HIGH** (`×3`)	`Dockerfile.prod`, `Dockerfile.test_local`, `Dockerfile.test_pypi`	`Specify at least 1 USER command in Dockerfile with non-root user as argument`	Create unprivileged `drawlib` user (`UID 10001`), store Playwright browsers and pre-downloaded assets under world-readable `/opt/` paths, and set `USER drawlib` before `WORKDIR /workspace`.
`DS-0026`	**LOW** (`×3`)	`Dockerfile.prod`, `Dockerfile.test_local`, `Dockerfile.test_pypi`	`Add HEALTHCHECK instruction in your Dockerfile`	Informational for ephemeral batch CLI/test containers (`CMD ["drawlib", "--help"]`), which exit upon completion rather than running a long-lived network daemon; document Trivy ignore or add `HEALTHCHECK NONE`.

Running `Dockerfile.prod` as non-root (`USER drawlib`) is particularly impactful for Drawlib because both embedded Python diagram `exec()` blocks and Playwright's headless Chromium

browser execute inside the container process tree.

Secret Scanning & `.dockerignore` Isolation (`SEC-03`)

- **Zero Hardcoded Source Secrets (`PASS`)**: Both AST literal entropy analysis (`AstFacts.hardcoded_secrets = []`) and Trivy repository scanning confirmed **zero API keys, credentials, or tokens** across all `424` tracked source files and `150` test files.
- **Default-Deny `.dockerignore` (`PASS`)**: Root `.dockerignore` begins with `*` (deny all) and explicitly allowlists only `!pyproject.toml`, `!src/`, and `!tests/`, guaranteeing local developer files can never leak into built Docker layers.
- **Untracked Local `.env` PyPI Token (`1` Redacted Workspace Finding)**: Trivy's local filesystem scan detected `pypi-upload-token` at `.env:1` (`PYPI_TOKEN=***`). Verification confirmed `.env` is excluded by `.gitignore:159` and `git ls-files .env` is empty (never committed to Git history). Migrating release publishing in `tools/dcli/pypi/` to **GitHub Actions OIDC Trusted Publishing** (`id-token: write`) eliminates the need to store static PyPI tokens on developer workstations.

6.3 AST Dangerous Code Sinks (`CWE-94`) & API Attack Surface

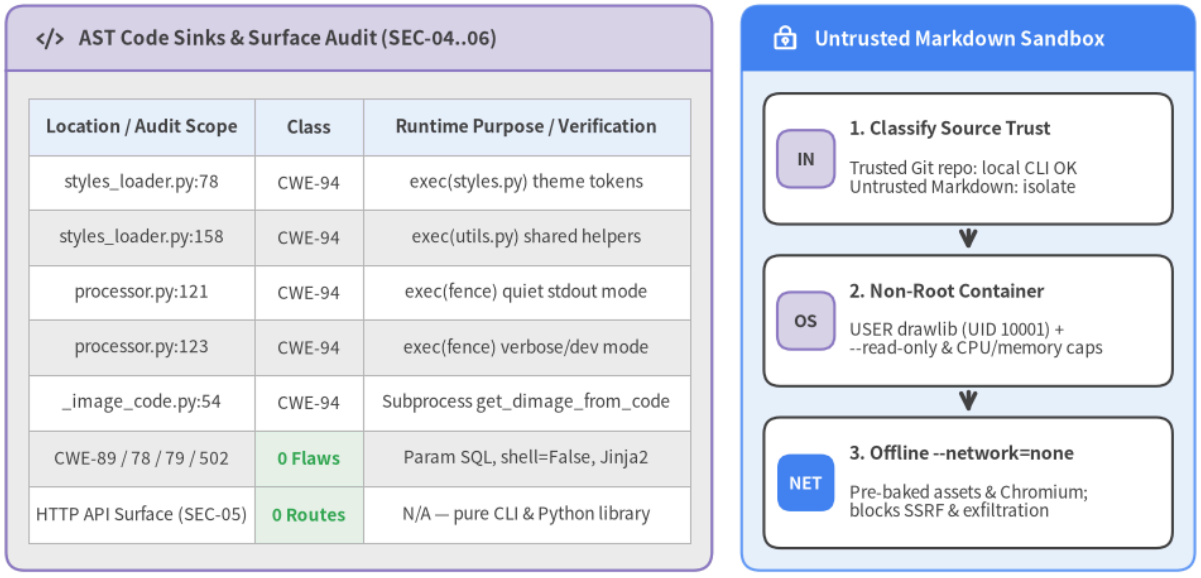


Figure 6.2 — Complete CWE-94 exec() sink inventory, zero injection/API surface verification, and recommended container sandbox boundary for untrusted Markdown.

Detailed Assessment of `CWE-94` Sites, Injection Classes & API Endpoints

1. **Five Intentional `CWE-94` (`exec`) Sinks (`SEC-04 WARN`)**:
   - `src/drawlib/_builder/_common/styles_loader.py:78` (`load_styles`) and `styles_loader.py:158` (`load_utils`) compile and execute project-level `styles.py` and `utils.py` files so documentation projects can customize `Styles` / `Colors` and register reusable drawing macros.
   - `src/drawlib/_builder/doc_builder/processor/processor.py:121` (quiet stdout-redirected mode) and `processor.py:123` (`verbose` / `developer` mode) execute ```` ```drawlib ```` fenced Python blocks inside Markdown files while temporarily monkey-patching `save()` as a controlled no-op.
   - `src/drawlib/_core/l4_canvas/_image_code.py:54` (`_worker_render_code`) executes nested drawing scripts inside an isolated `multiprocessing` child process

for `get_dimage_from_code(code, timeout=...)`.

- **Architectural Verdict**: Just like Jupyter notebooks, Quarto, or Sphinx code-execution directives, executing Python code is the core value proposition of Drawlib (*"Illustration as Code"*). Python-level `ast` allowlisting or `__builtins__` restriction is notoriously bypassable via object-graph traversal and should **not** be relied upon as a security boundary. Instead, Drawlib should explicitly document this trust boundary in `SECURITY.md` and CLI help: run `drawlib build` / `drawlib show` directly only on trusted sources, and wrap untrusted third-party Markdown rendering inside an ephemeral container (`--user 10001:10001 --network=none --read-only --pids-limit=128 --memory=1g`) built with `--full` pre-cached assets.

2. **Zero Injection or Deserialization Vulnerabilities (`PASS`)**:
   - `CWE-89` **(SQL Injection = `0`)**: `BuildImageCache` and `CliImageCache` in `src/drawlib/_builder/_common/cache.py` use strictly parameterized `?` SQLite queries on static table schemas.
   - `CWE-78` **(OS Command Injection = `0`)**: Subprocess execution in `src/drawlib/_builder/_common/browser.py:210` passes explicit argument arrays (`[str(exec_path), str(cli_path), *_INSTALL_ARGS]`) without `shell=True`.
   - `CWE-79` / `CWE-502` **(XSS / Unsafe Deserialization = `0`)**: HTML templates rely on `Jinja2` + `MarkupSafe` escaping, manifests parse via stdlib `tomllib`, and caches store raw PNG/SVG bytes plus SHA-256 digests (zero `pickle` or `yaml.load` usage).
3. **API Endpoint Surface (`SEC-05` / `SEC-06`: `api_endpoints = []`, `not_applicable`)**:
   - Drawlib's runtime archetype is `["cli", "library"]` with **zero production HTTP/gRPC service endpoints**. The `drawlib serve` subcommand launches a strictly local static preview server bound to `127.0.0.1` by default for interactive authoring and `--check` link validation.

6.4 OpenSSF Supply-Chain Posture & Governance Checklist

Drawlib scored **86/100 (Grade A)** on `supply_chain_audit` and **89/100 (Grade A)** on `repo_structure_audit`, combining strong cryptographic asset integrity and least-privilege CI

workflows with straightforward governance additions (`SECURITY.md` , Dependabot, and commit-SHA Action pins):

Supply-Chain Control	Current Repository State	Evidence Location	Recommended Hardening
CI Workflow Token Permissions	**PASS**	`.github/workflows/{quality,ut_github,ut_pypi,ut_template}.yml`	All `4` workflows declare explicit top-level `permissions: contents: read` ; retain least-privilege defaults when adding OIDC release jobs.
External Binary Asset Integrity	**PASS**	`src/drawlib/_core/l3_external/_package.py` & `src/drawlib/_release_assets.py`	Every lazy-downloaded font, icon, and GeoJSON archive embeds a compiled-in **SHA-256 digest** verified before zip extraction alongside `timeout=60` HTTPS fetches.
Runtime Dependency Specifiers	**PASS**	`pyproject.toml` (`dependencies` , `13` packages)	`100%` of runtime dependencies specify explicit SemVer lower bounds (`unpinned_dependencies = []`), cleanly separated from `[dependency-groups].dev` .
Lockfile Strategy (`uv.lock`)	**INFO**	`uv.lock` & `.gitignore:182`	`uv.lock` is generated locally for reproducible audits while `.gitignore` excludes it so CI tests flexible library resolution across Python `3.11–3.13` ; document this intentional library policy or commit `uv.lock` alongside a separate `--resolution=lowest-direct` CI job.
GitHub Actions Pinning	**WARN** (`P2`)	`.github/workflows/quality.yml:20,23,28`	Steps currently reference mutable major tags (`actions/checkout@v4` , `astral-sh/setup-uv@v5` , `actions/setup-python@v5`); pin to full 40-char commit SHAs with version comments.
Vulnerability Disclosure Policy	**MISSING** (`P1/P2`)	Repository root (`SECURITY.md` absent)	Add `SECURITY.md` enabling GitHub Private Vulnerability Reporting, supported version policy (`0.3.x`), and the `CWE-94` `exec()` container-sandbox guidance.
Automated Dependency Updates	**MISSING** (`P1/P2`)	`.github/dependabot.yml` absent	Configure weekly `.github/dependabot.yml` updates for `uv` / `pip` and `github-actions` so future `pillow` or Action advisories open automated PRs immediately.